Tarlac State University
COLLEGE OF COMPUTER STUDIES

Case Study: Simulation of CPU Scheduling Algorithms

A CASE STUDY Presented to

the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City, Tarlac State University

In Partial Fulfillment

of the Requirements for the Course

Operating Systems

Submitted By:
Vergara, Ma. Carmela Veronica A.

Submitted To:
Ma'am Jo Anne G. Cura

May 15, 2026

I. Introduction

1.1 Overview CPU Scheduling.

CPU scheduling is a basic principle of the operating systems that defines how processes are allocated to the CPU to run. A process is a program that is being executed and takes CPU time to execute. The operating system should determine what process will be using the CPU at any particular time since there can be several processes running simultaneously. This process of making decisions is referred to as CPU scheduling and is significant in the efficient use of system resources as well as reducing the amount of idle time by the CPU (Unstop, 2026).

CPU scheduling enables multiple processes to utilize the CPU efficiently in a multitasking environment by sorting their execution sequence with various scheduling algorithms. These algorithms include First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time (SRT), and Round Robin (RR) that aim to enhance the performance, responsiveness, and equity of a system. Through the use of proper scheduling techniques, the operating system is able to effectively control the processes running and the stability of the entire system (GeeksforGeeks, 2026; Mathur, 2022).

1.2 Purpose of the Case Study

This case study aims to create and develop a program to simulate different CPU scheduling algorithms as a real-life example of process management in operating systems. The study seeks to illustrate the impact of various scheduling methods on the execution of a process, especially the waiting time and turnaround time. This will assist in determining the impact of scheduling decisions on system performance, efficiency and responsiveness. Also, the case study will help in closing the gap between theory and practice. Through the use of scheduling algorithms in a working program, one is able to gain a better insight into the way operating systems handle a number of processes that are competing to use the CPU resources in real-world situations.

1.3 Program Objectives

The primary purpose of this program is to emulate basic CPU scheduling algorithms and compare their performance in terms of the standard scheduling metrics.

In particular, the program is to:

- Use various CPU scheduling algorithms such as FCFS, SJF (non-preemptive), SRT (preemptive), Round Robin, Priority Scheduling and Priority Scheduling with Round Robin.
- Accept user input like number of processes, arrival time, burst time, priority values, and time quantum.
- Note down a Gantt chart to show the process schedule.
- Calculate and display Waiting Time (WT) and Turnaround Time (TAT) of all processes. Also, the Average of Waiting time and Turnaround time.
- Compare the performance of different scheduling algorithms in terms of efficiency, fairness and responsiveness.
- Demonstrate the improvement of CPU utilization, waiting time, and system throughput by CPU scheduling.

II. Body

2.1 First Come, First Serve (FCFS)

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm where processes are executed strictly in the order they arrive in the ready queue. It is based on the principle of First In, First Out

(FIFO), the process first to arrive is first executed without interruption of any kind. When a process is initiated it will be executed until it is completed irrespective of the time taken. Due to this characteristic, FCFS is considered a non-preemptive scheduling algorithm. It is commonly applied in batch processing systems like payroll systems and billing operations, where the tasks are processed in a sequence without the involvement of a user. Although FCFS is simple to apply and equitable in terms of arrival order, it may lead to long wait times particularly when a long process is run before shorter processes. This causes the issue of the convoy effect where shorter processes are held up by longer processes, decreasing the efficiency of the entire system (Awobode, 2022; TutorialsPoint, n.d.).

Type: Non-Preemptive

Advantages:

- Simple and easy to implement
- Fair scheduling based on arrival time
- Low overhead due to minimal context switching

Disadvantages:

- High average waiting time
- Convoy effect reduces system efficiency
- Not suitable for interactive systems

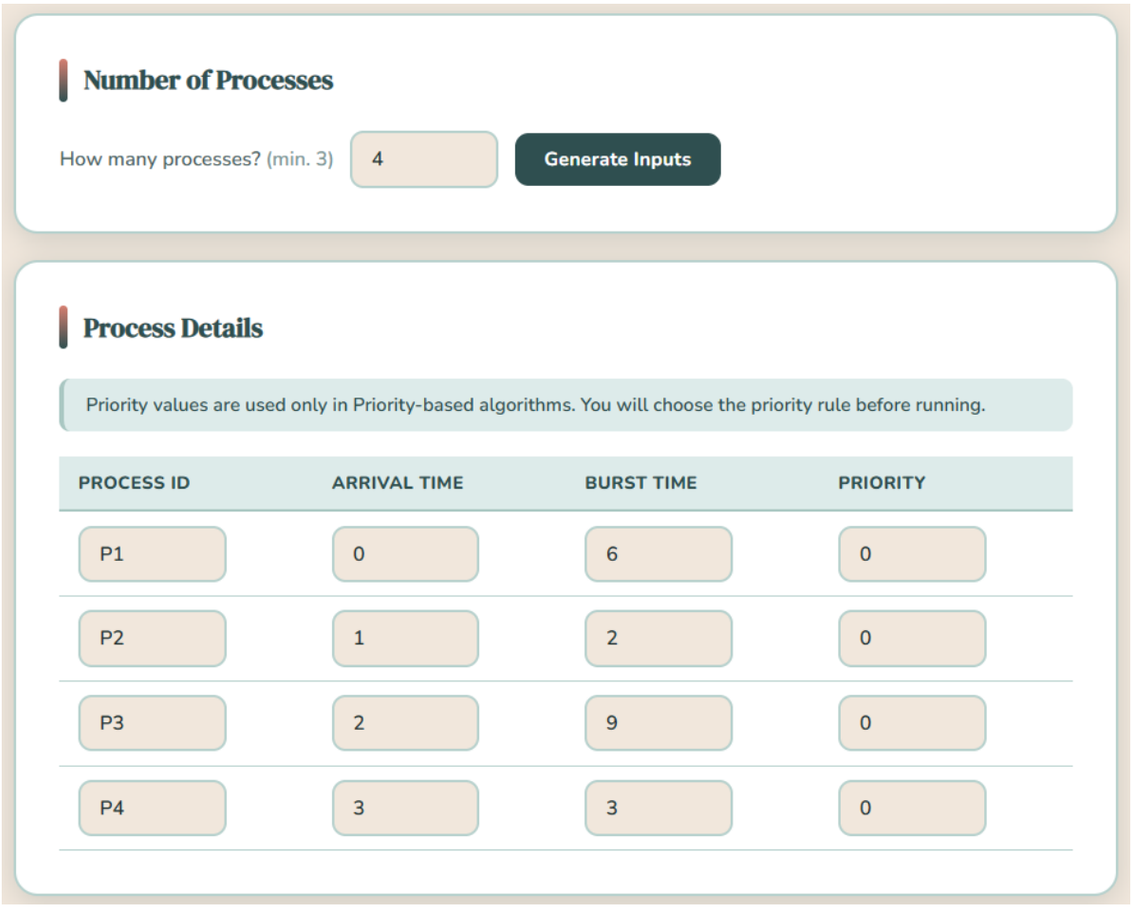

Figure 1.1: *FCFS Input Parameters (Test Case 1)*

Figure 1.2: *FCFS Algorithm Selection (Test Case 1)*

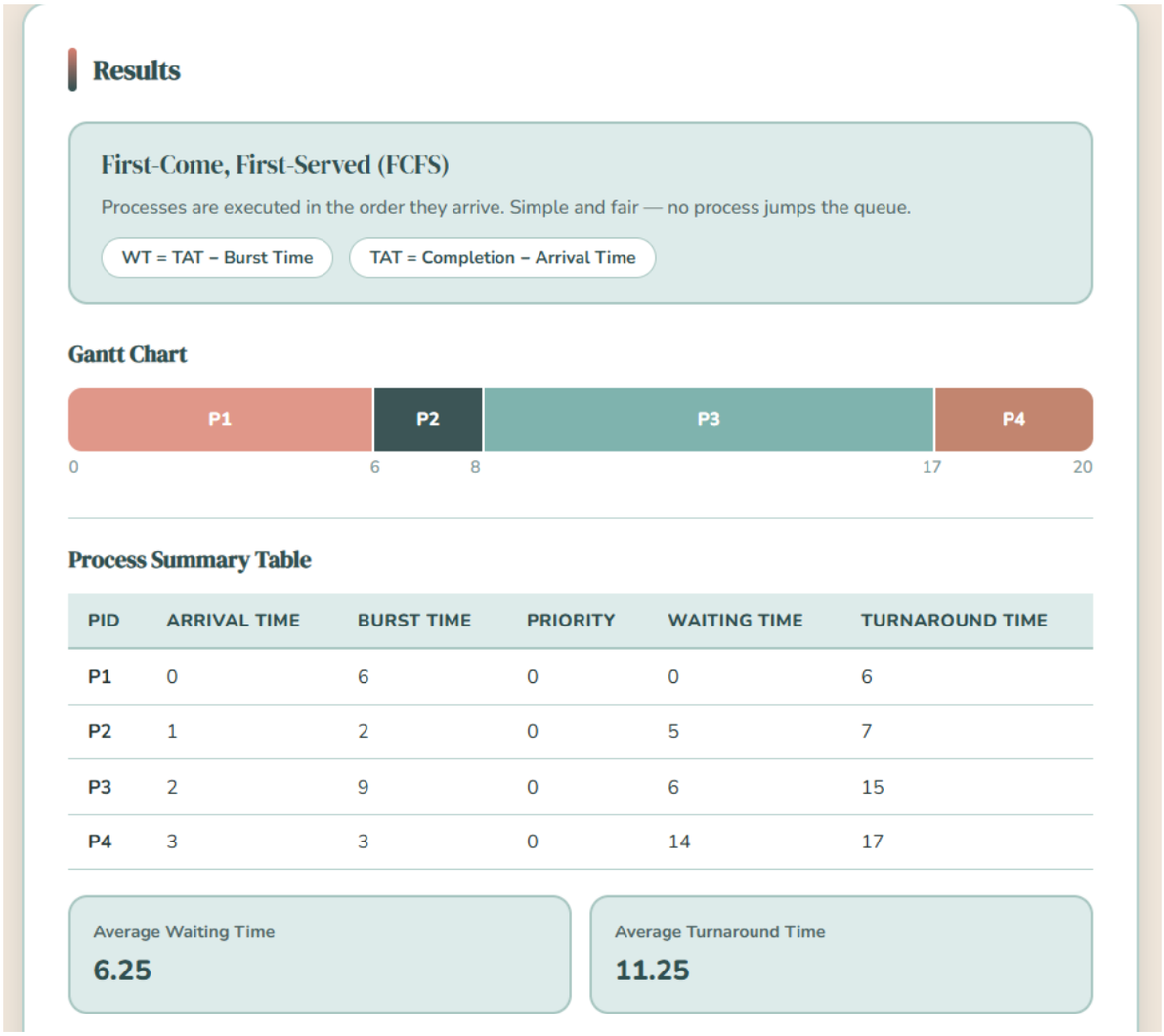

Figure 1.3: *FCFS Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

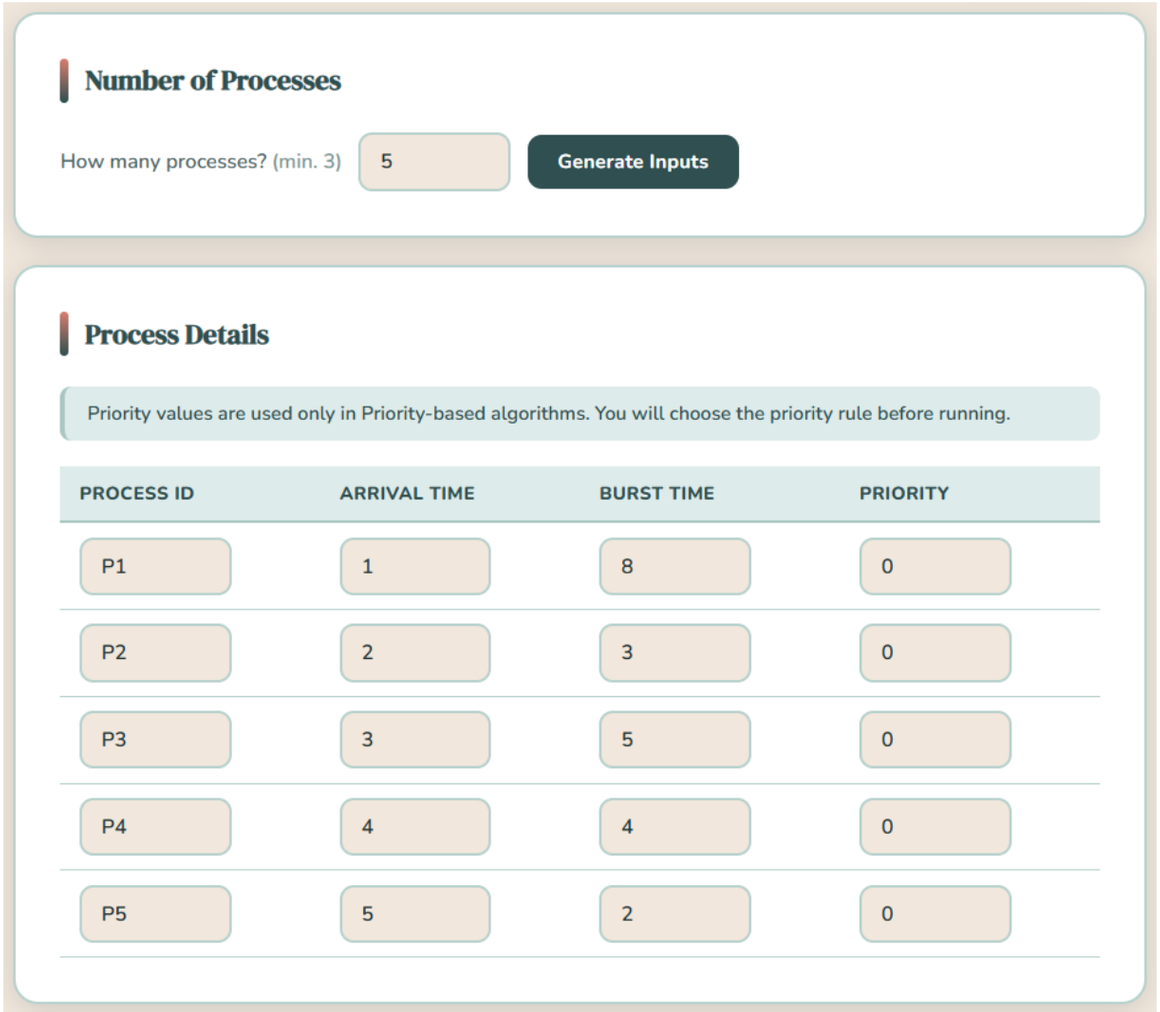

Figure 2.1: *FCFS Input Parameters (Test Case 2)*

Figure 2.2: *FCFS Algorithm Selection (Test Case 2)*

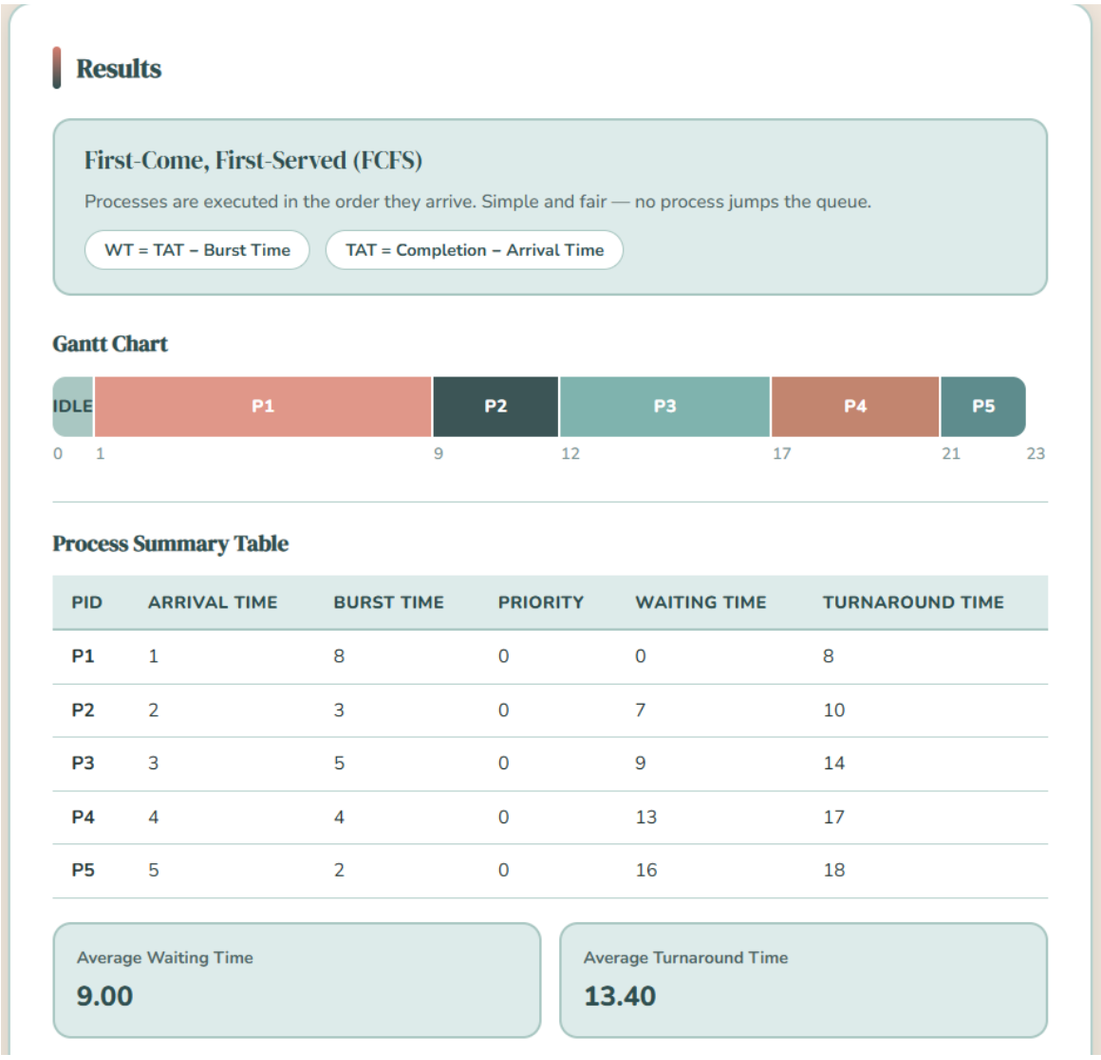

***Figure 2.3**: FCFS Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.2 Shortest Job First (SJF) – Non-preemptive

Shortest Job First (SJF) is a scheduling algorithm that chooses the process with the least burst time to be executed. This algorithm is primarily aimed at reducing the average waiting time, prioritizing the shortest processes, in preference to the longest ones. In non-preemptive version, once a process begins to be run, it is not interrupted until it completes. SJF is best suited in the reduced average waiting and turnaround time where the burst time of each process is known. But in real world systems, burst time cannot be predicted with much accuracy and hence it is not applicable in practice. Also long processes can be starved when shorter processes are continuously arriving. Regardless of these constraints, SJF can be successfully used in batch systems with predictable execution times (GeeksforGeeks, 2026).

Type: Non-Preemptive

Advantages:

- Minimizes average waiting time
- Efficient for batch processing systems
- Provides optimal turnaround time (theoretical)

Disadvantages:

- Requires prior knowledge of burst time
- Can cause starvation for longer processes
- Difficult to implement in real-time systems

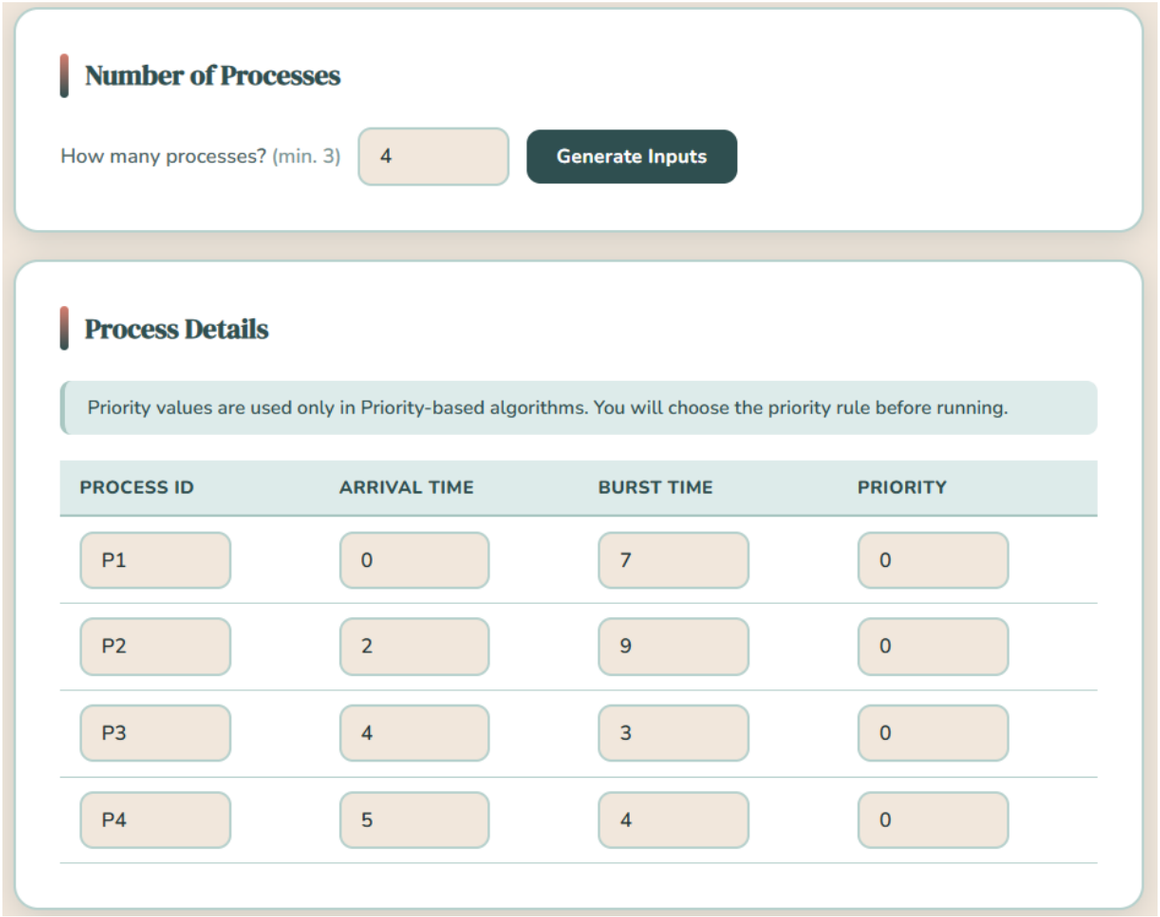

Figure 3.1: *SJF Input Parameters (Test Case 1)*

Figure 3.2: *SJF Algorithm Selection (Test Case 1)*

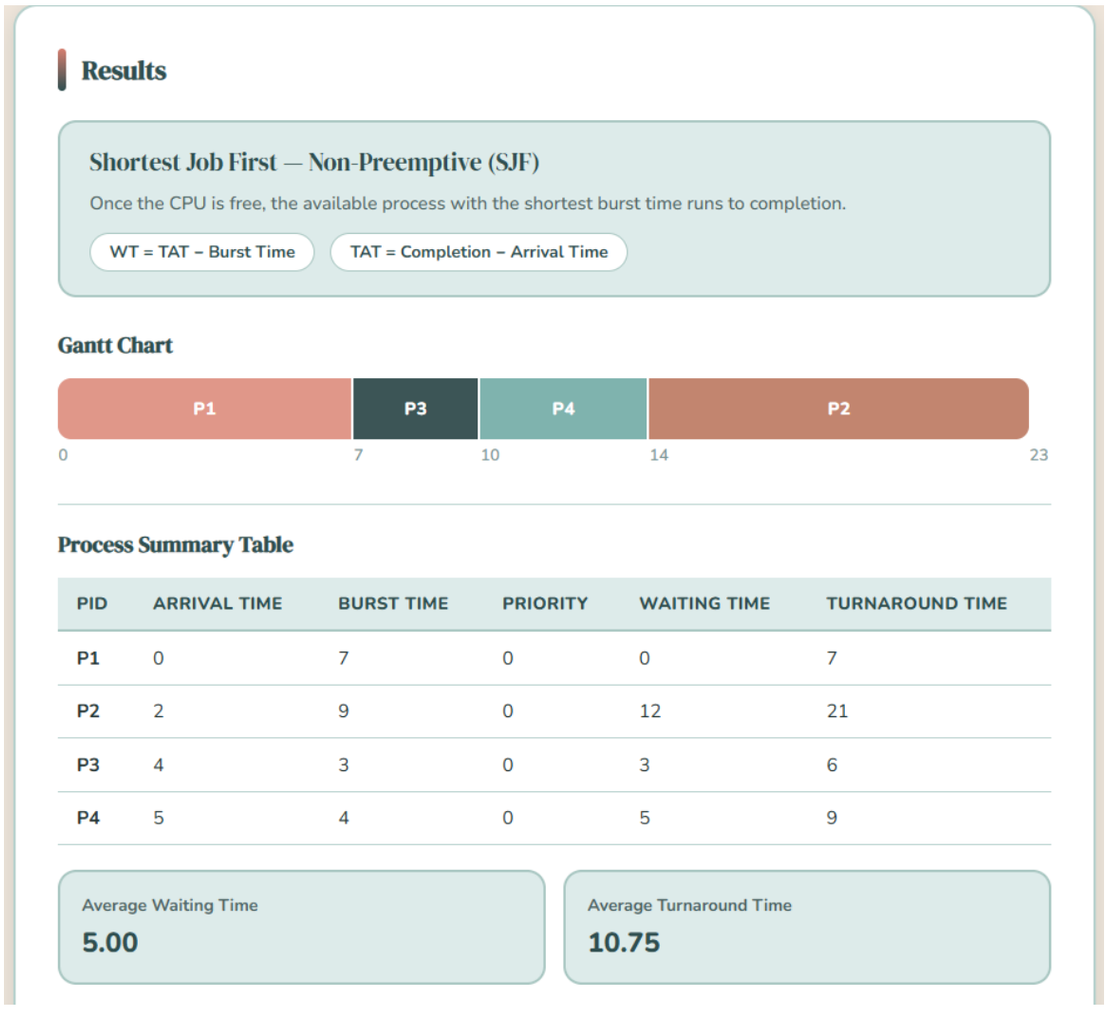

Figure 3.3: *SJF Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

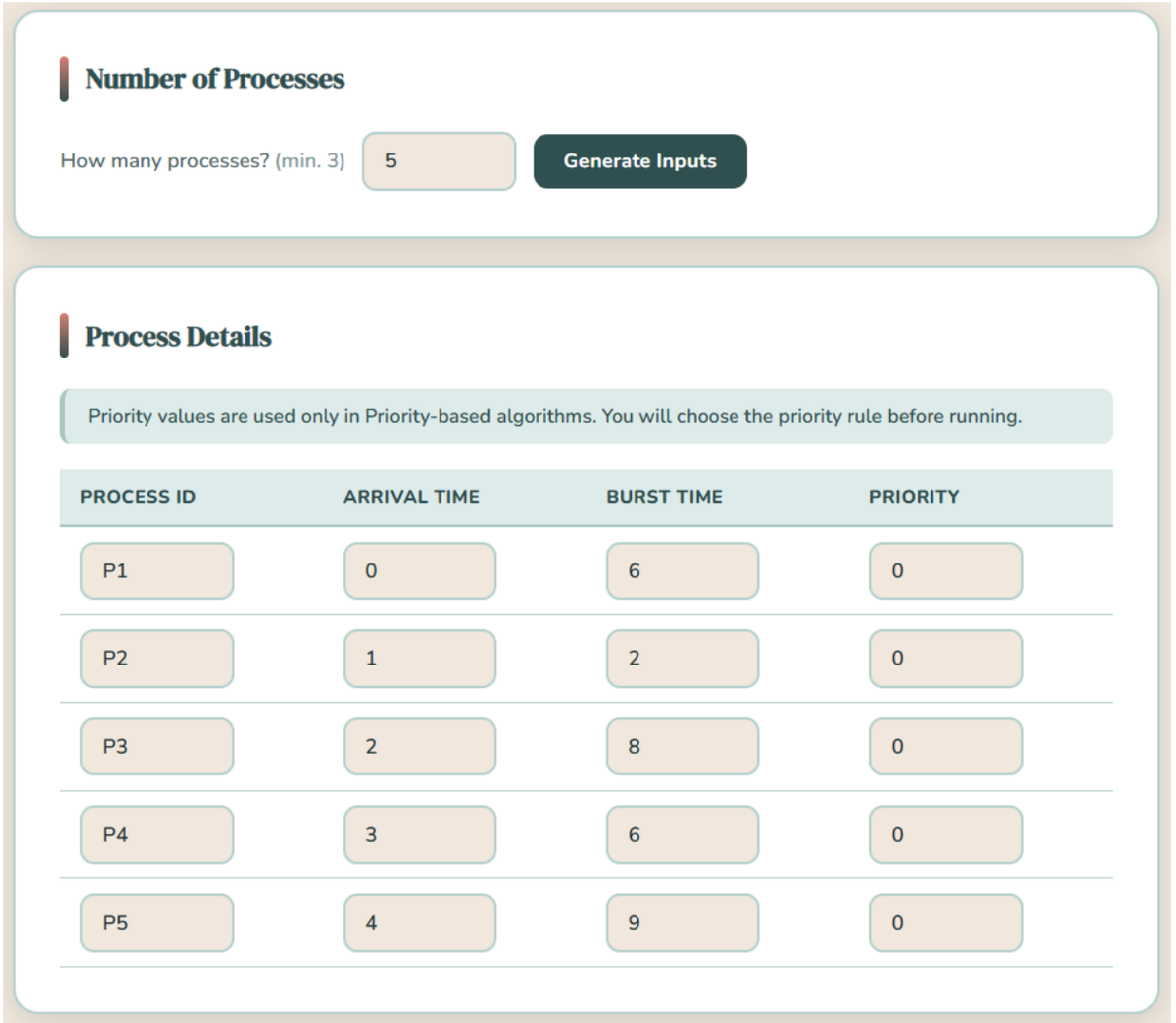

Figure 4.1: *SJF Input Parameters (Test Case 2)*

Figure 4.2: *SJF Algorithm Selection (Test Case 2)*

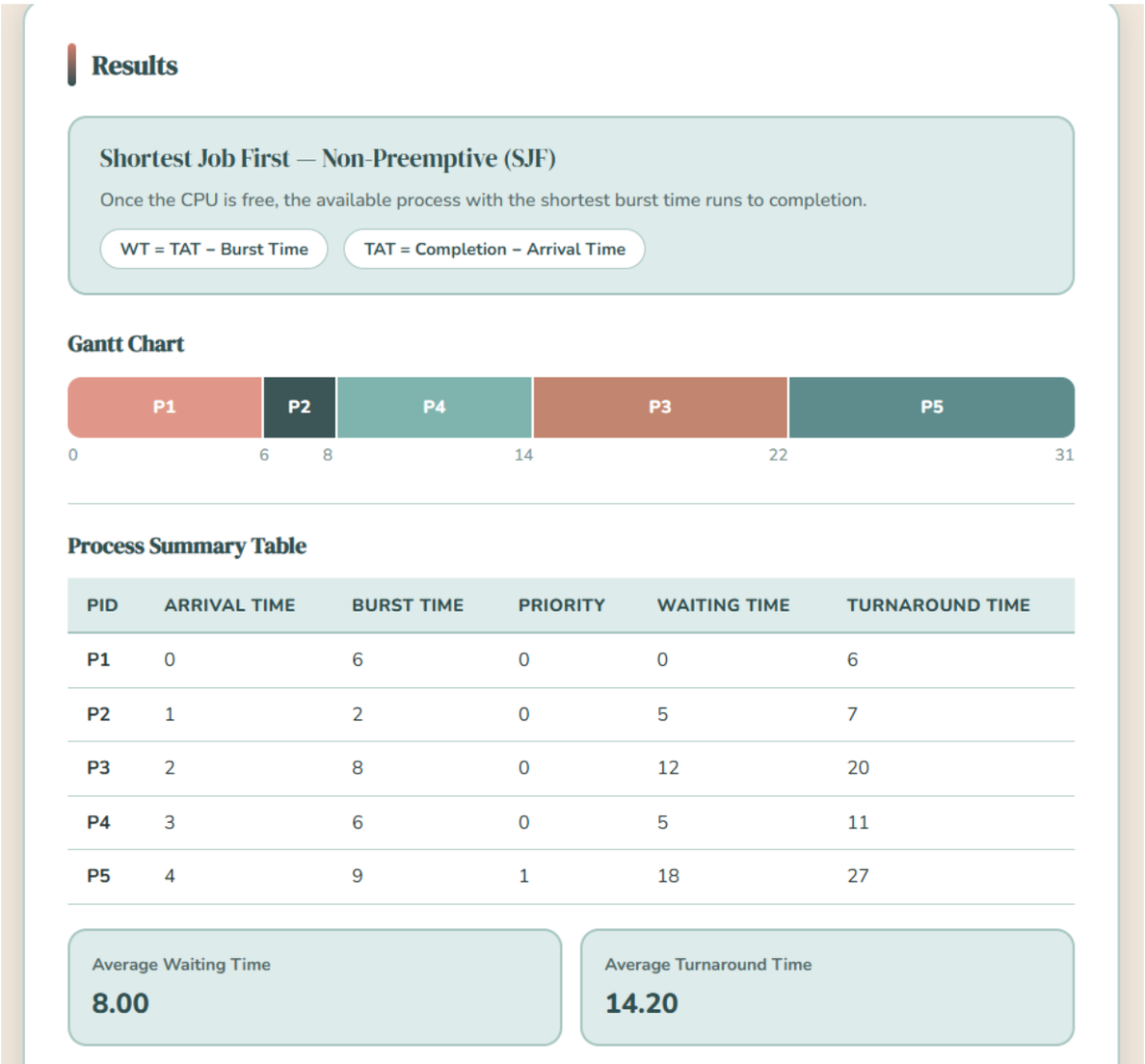

Figure 4.3: *SJF Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.3 Shortest Remaining Time (SRT) – Preemptive

The preemptive version of Shortest Job First algorithm is known as Shortest Remaining Time First (SRTF). The CPU is always allocated to the process whose remaining execution time is the shortest in this method. When a new process comes with less remaining time than the one currently running, the CPU will instantaneously change to the new process. This makes sure that the shortest tasks are done within the shortest time possible thereby increasing the responsiveness of the system. But due to the frequent interruption of running processes by SRTF, it causes many context switches leading to system overhead. Although it has an advantage over non-preemptive SJF in terms of waiting time, it is more difficult to manage and needs constant observation of process execution time (GeeksforGeeks, 2025).

Type: Preemptive

Advantages:

- Reduces average waiting time significantly
- Improves responsiveness for short processes
- More efficient than non-preemptive SJF

Disadvantages:

- High overhead due to frequent context switching
- Complex to implement
- Requires accurate tracking of remaining time

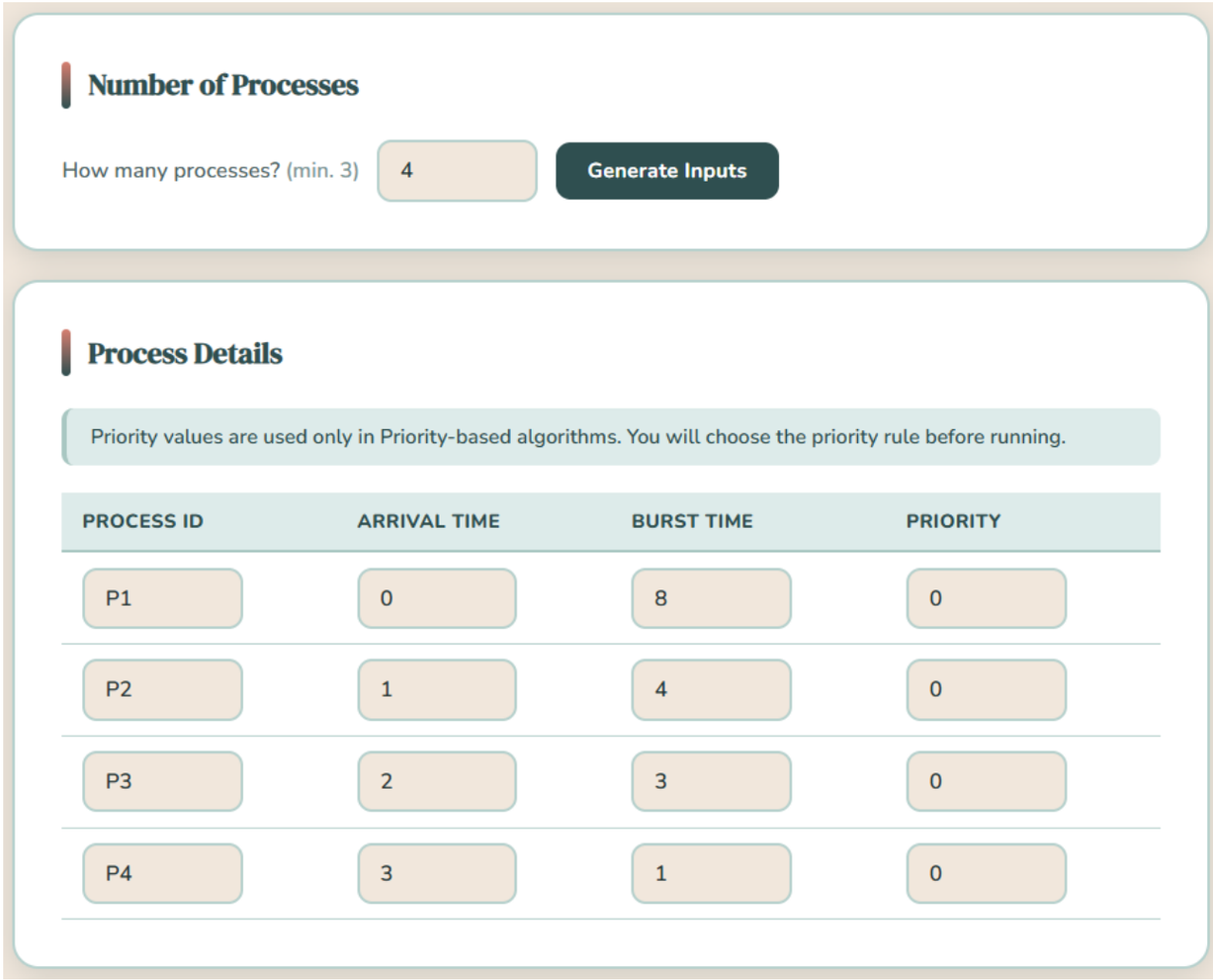

Figure 5.1: *SRT Input Parameters (Test Case 1)*

Figure 5.2: *SRT Algorithm Selection (Test Case 1)*

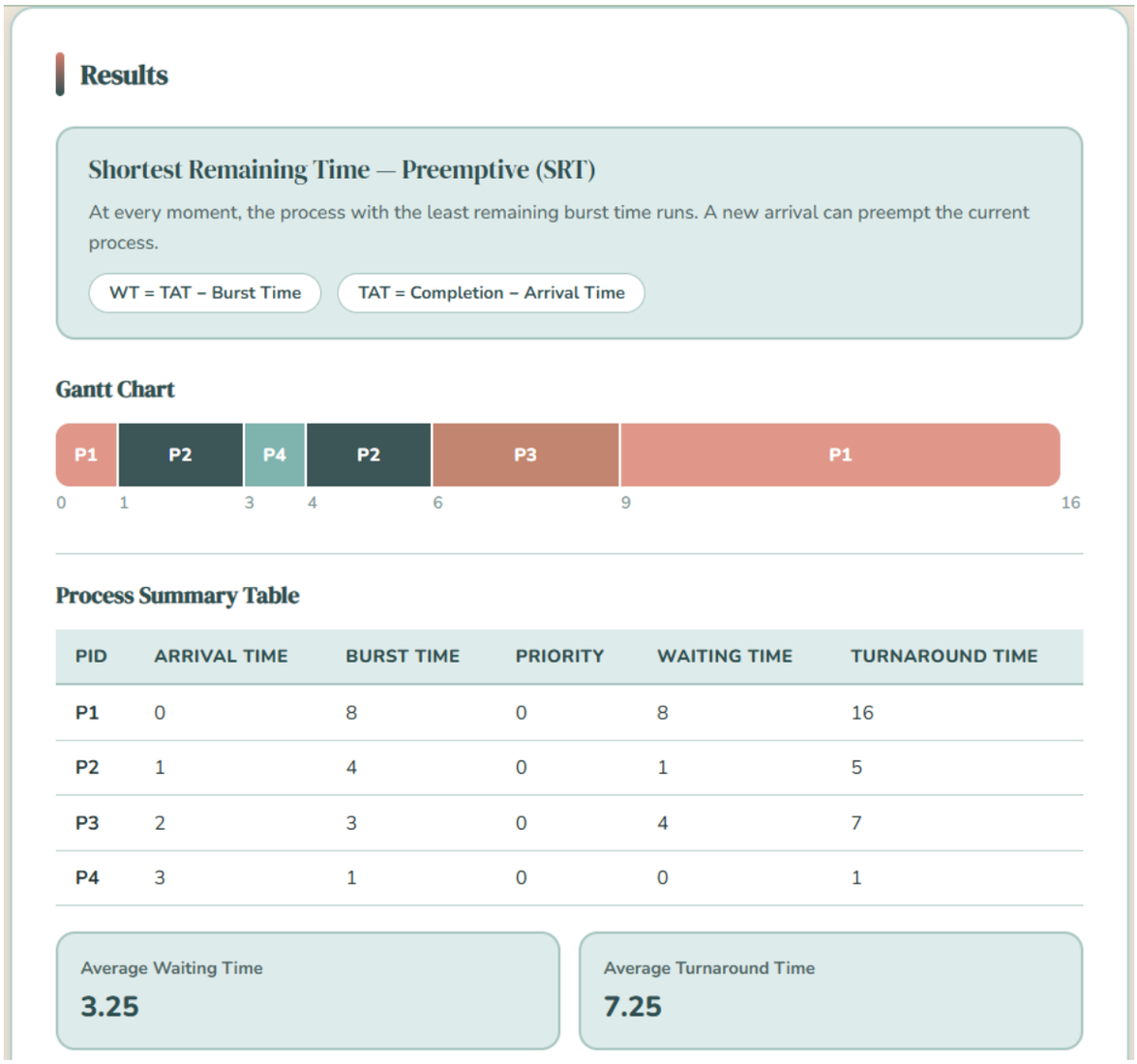

Figure 5.3: *SRT Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

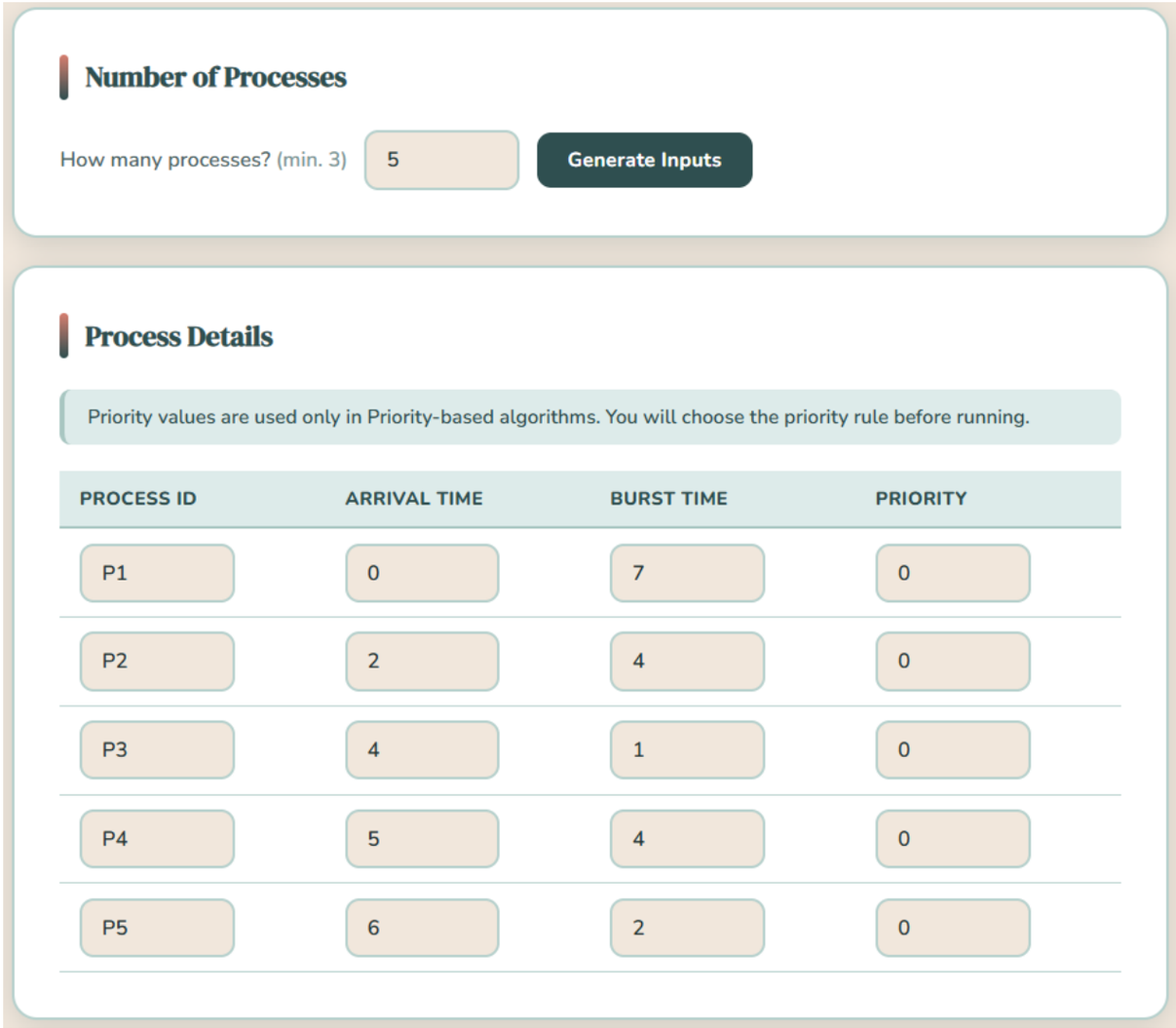

Figure 6.1: *SRT Input Parameters (Test Case 2)*

Figure 6.2: *SRT Algorithm Selection (Test Case 2)*

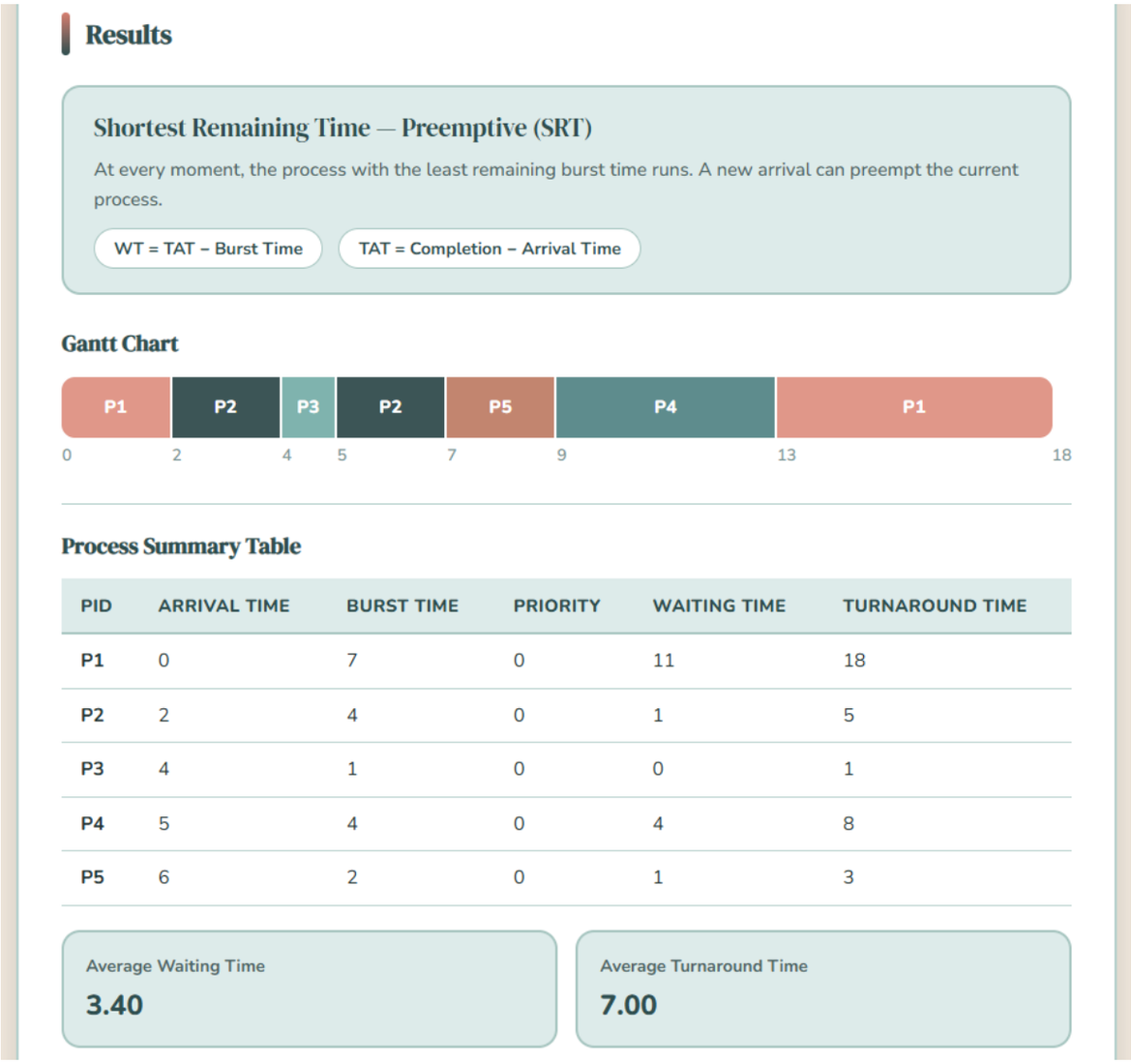

Figure 6.3: *SRT Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.4 Round Robin (RR)

Round Robin (RR) is a preemptive scheduling algorithm that is used in time-sharing systems. In this technique, every process is allocated a fixed time interval, called a time quantum and the CPU executes the processes in the ready queue. A process that cannot finish in its assigned time quantum is preempted and placed at the end of the queue, which enables other processes to run. This rotation goes on until all processes are done. The key benefit of Round Robin is that it is fair because every process gets the same amount of CPU time. It also enhances responsiveness of the system hence being appropriate in interactive systems. The size of the time quantum is however critical in the performance of the algorithm. A small quantum is equivalent to adding to the context switching overhead, whereas a large quantum is equivalent to FCFS (GeeksforGeeks, 2026).

Type: Preemptive

Advantages:

- Fair allocation of CPU time
- Suitable for time-sharing systems
- Good responsiveness

Disadvantages:

- High context switching overhead (small quantum)

- Poor performance if quantum is too large
- Efficiency depends on time quantum selection

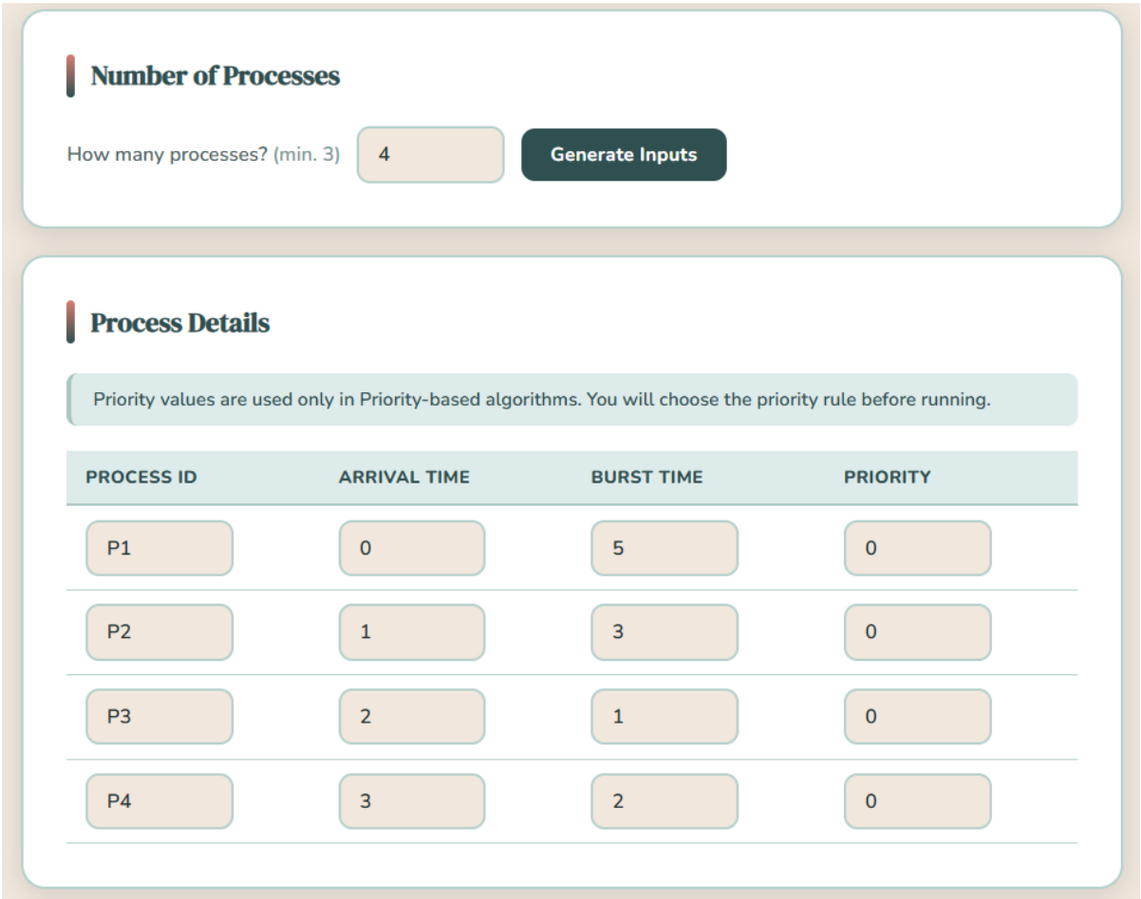

Figure 7.1: *Round Robin Input Parameters (Time Quantum =2, Test Case 1)*

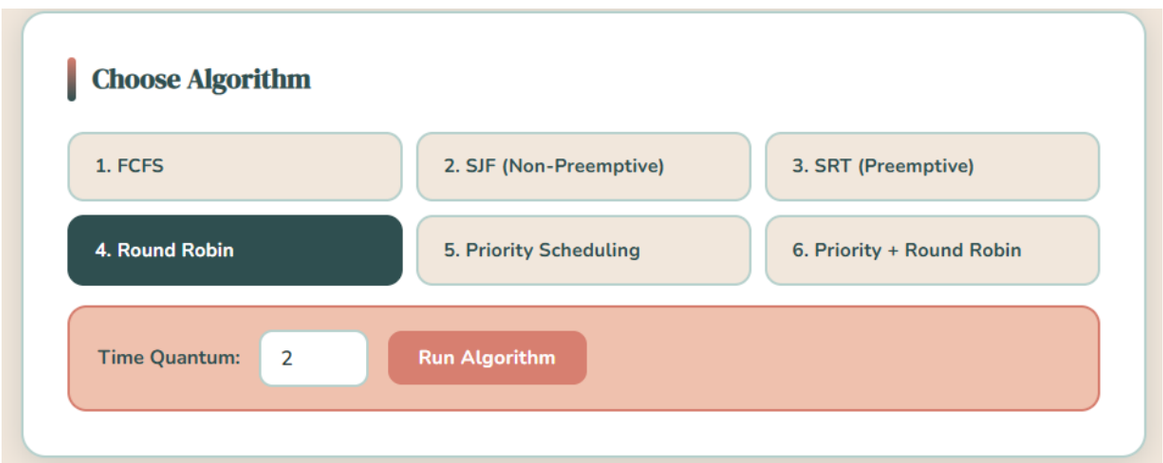

Figure 7.2: *Round Robin Algorithm Selection (Test Case 1)*

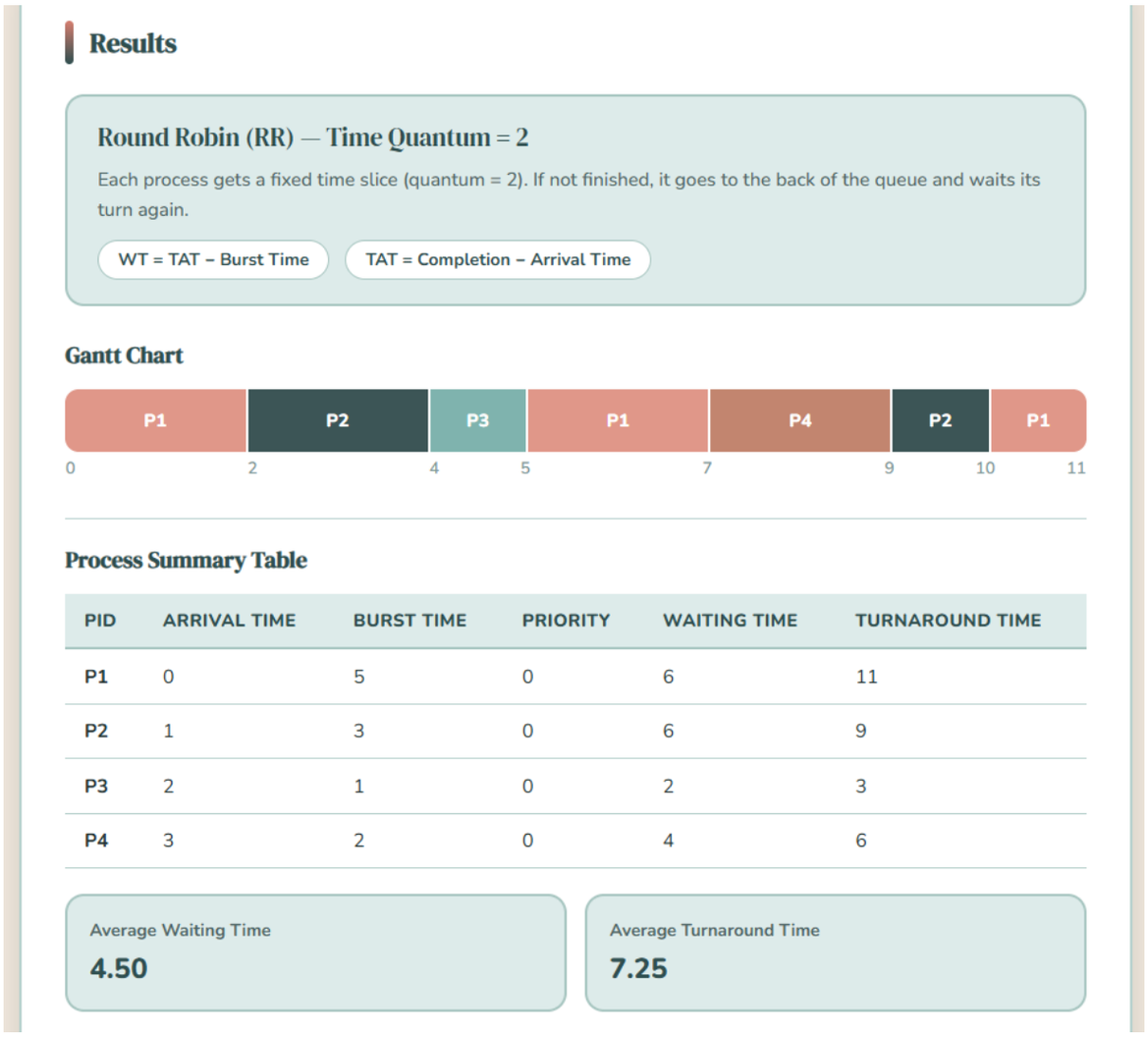

Figure 7.3: *Round Robin Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

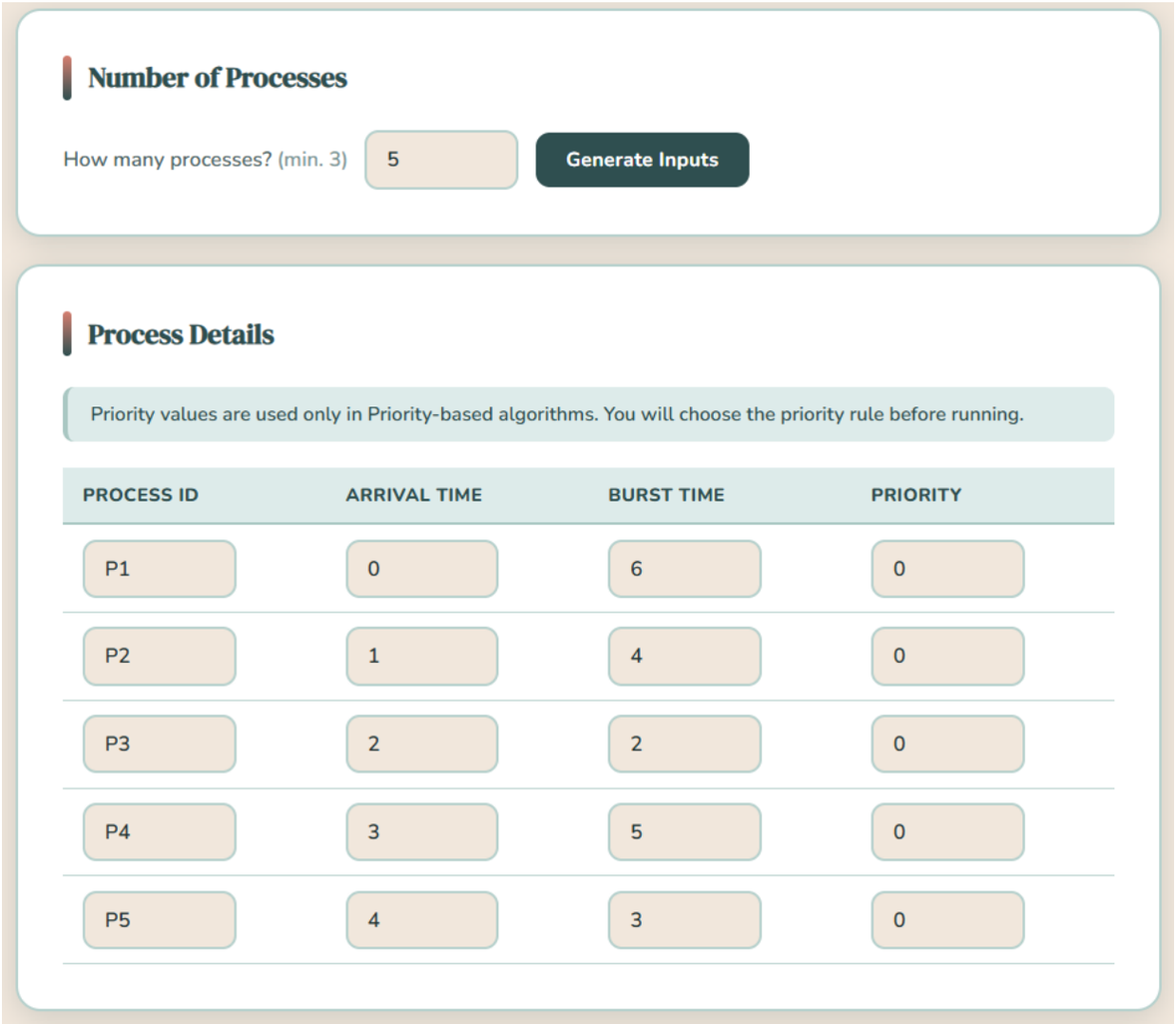

Figure 8.1: *Round Robin Input Parameters (Time Quantum = 2, Test Case 2)*

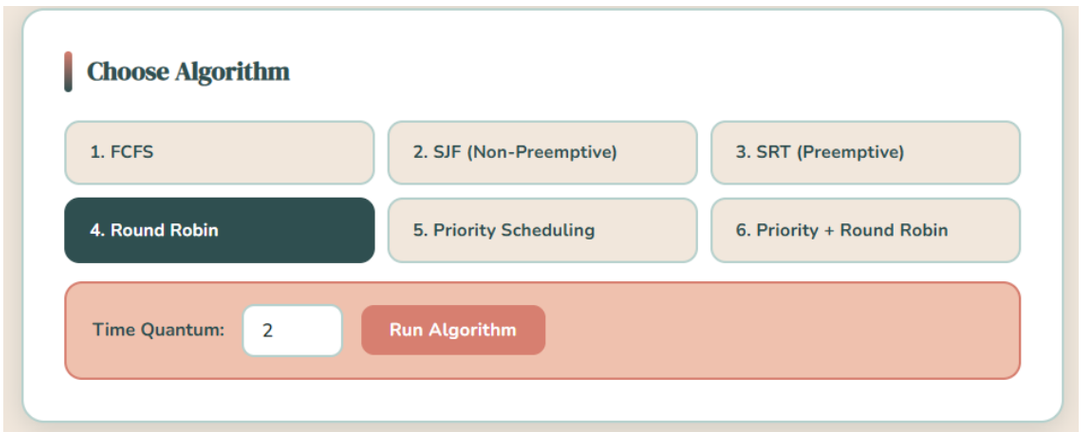

Figure 8.2: *Round Robin Algorithm Selection (Test Case 2)*

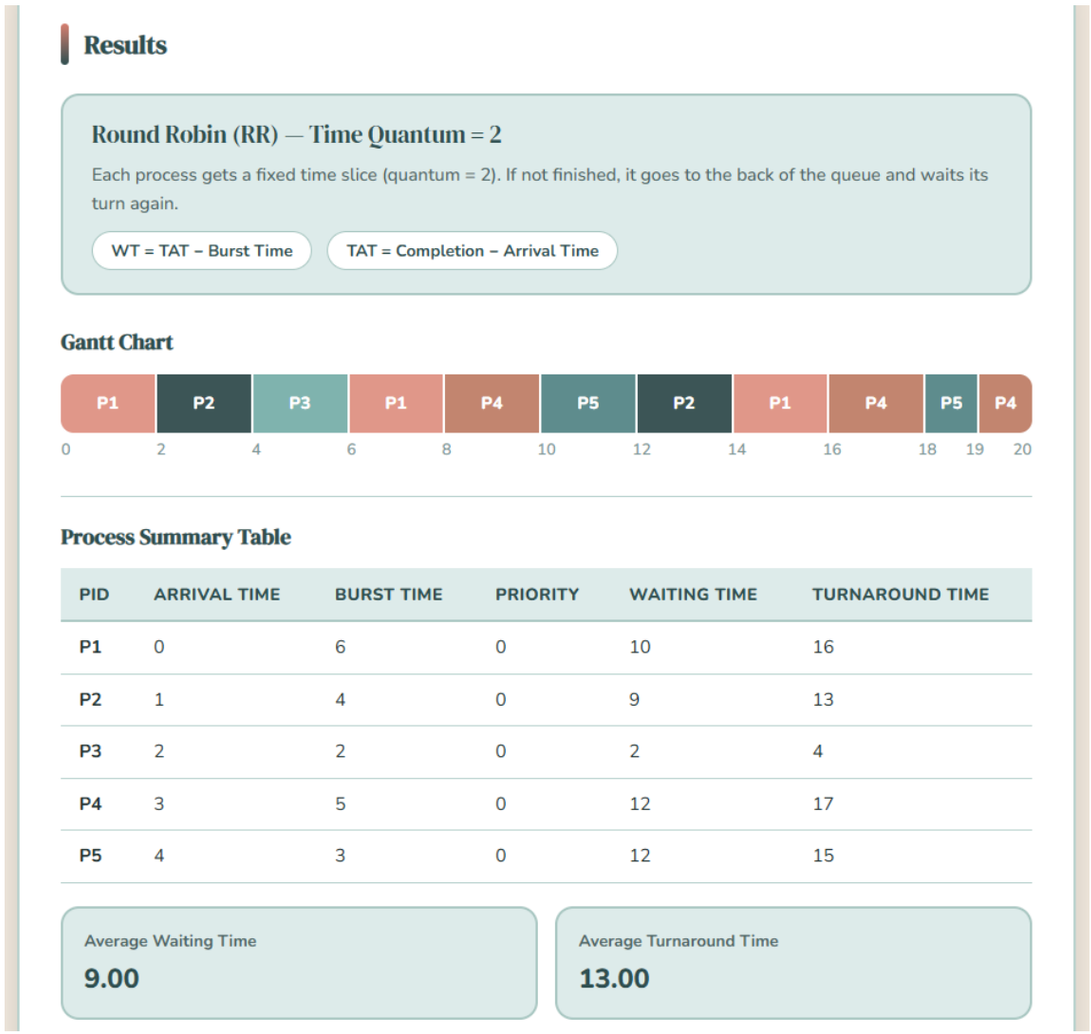

Figure 8.3: *Round Robin Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.5 Priority Scheduling (Non-Preemptive)

Priority Scheduling (Non-Preemptive) is a CPU scheduling algorithm in which processes are completed in the order of their assigned priority and once a process begins to run it will run to completion without being preempted. When the CPU is free, the scheduler will pick the process that has the highest priority in the ready queue. When a process with a higher priority arrives when there is a running process, it will not interrupt the running process but will wait until the CPU is free. Where there is an equal priority of processes, the algorithm adheres to FCFS approach. The approach is easy and effective in a system where the priority of tasks is well established, yet it can cause low-priority processes to starve (Mathur, 2024; Waraich, 2024).

Type: Non-Preemptive

Advantages:

- Simple and easy to implement
- Ensures important tasks are executed first
- Low overhead

Disadvantages:

- Starvation of low-priority processes
- Not suitable for dynamic systems
- Poor responsiveness for lower-priority tasks

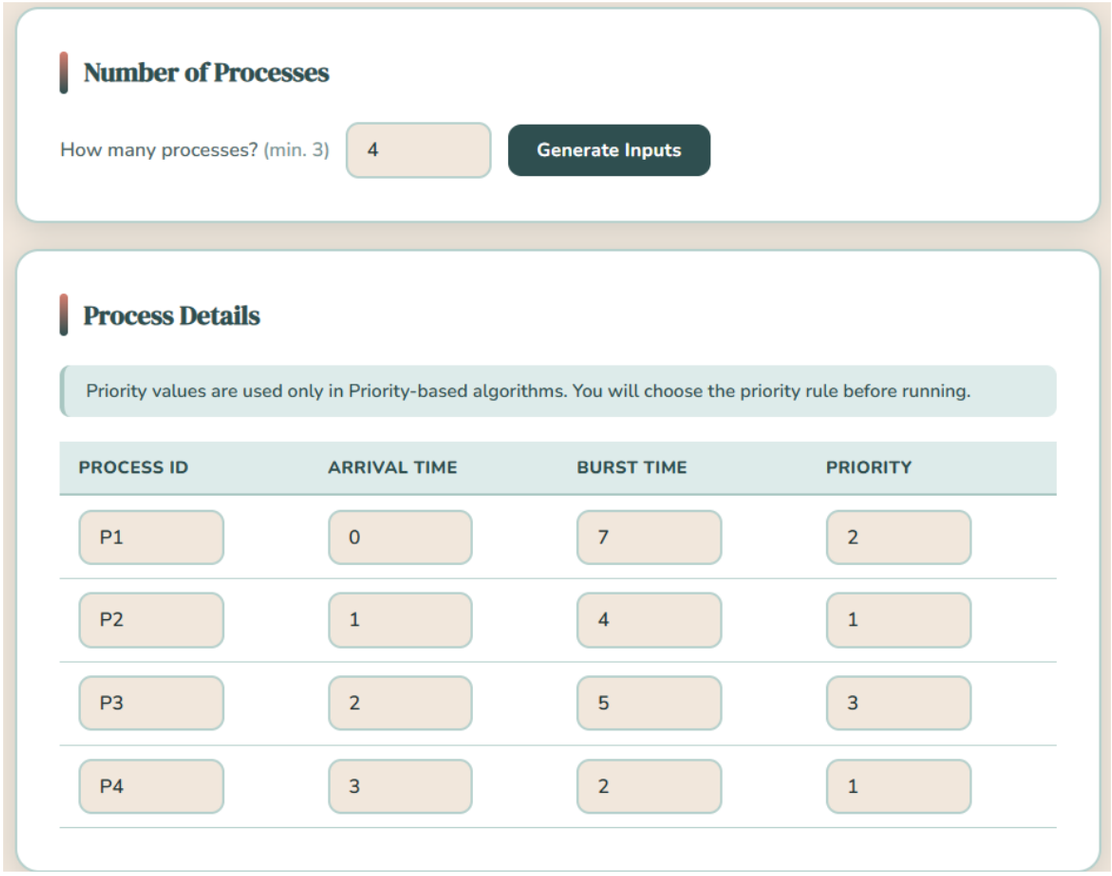

Figure 9.1: *Priority Scheduling (Non-Preemptive) Input Parameters (Test Case 1)*

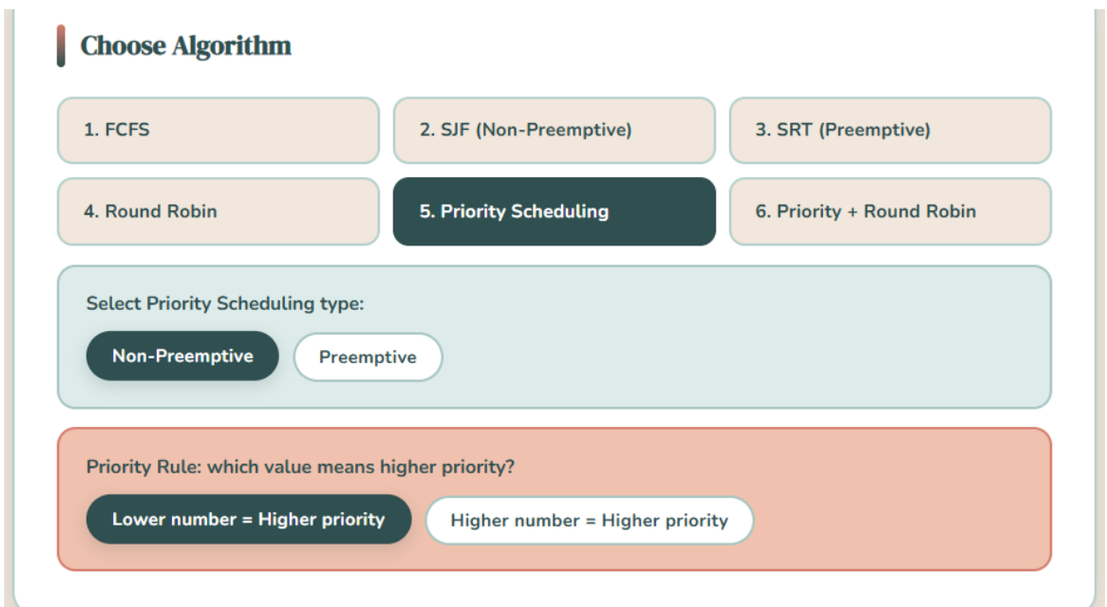

Figure 9.2: *Priority Scheduling (Non-Preemptive) Algorithm Selection (Test Case 1)*

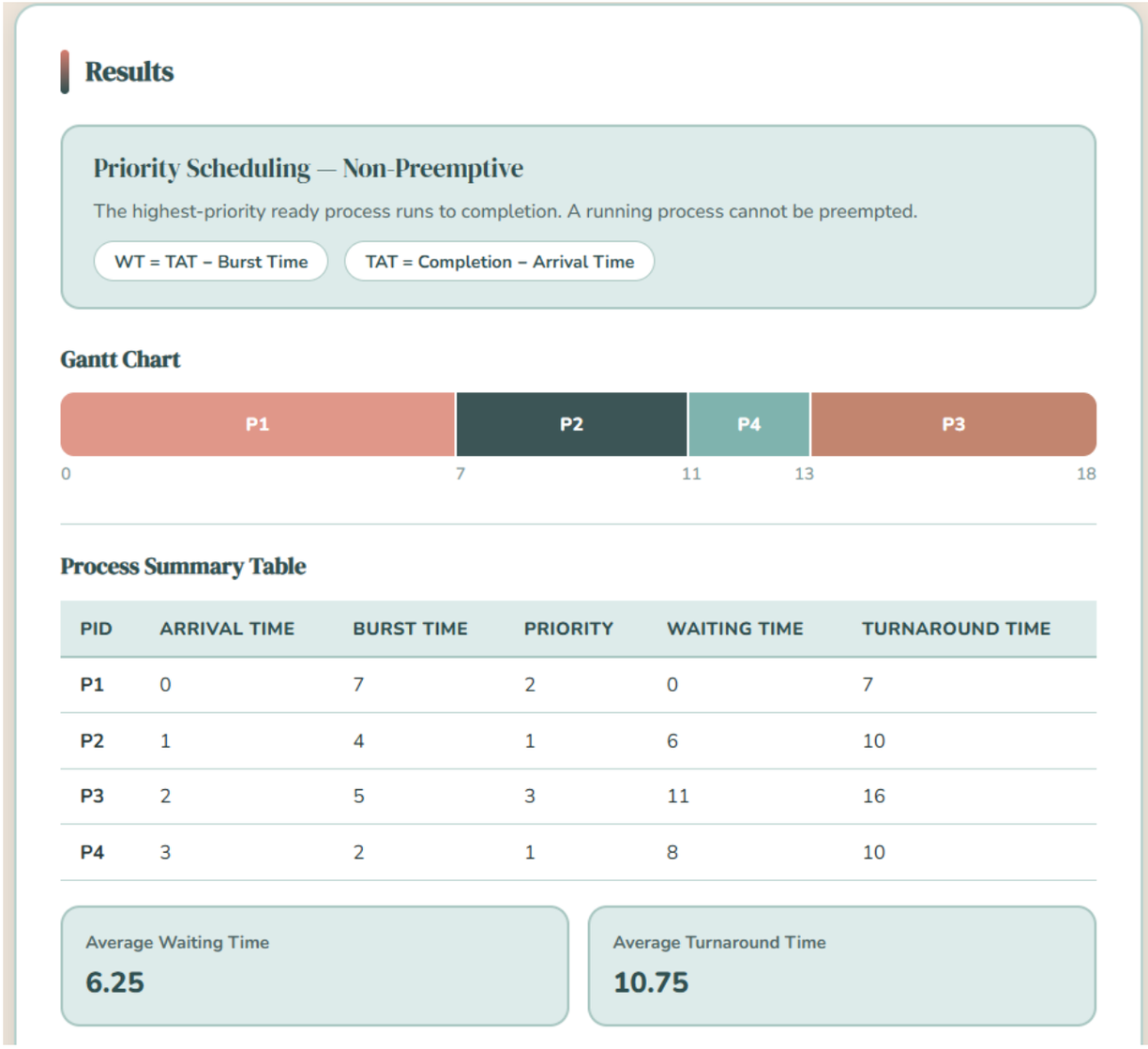

Figure 9.3: *PS (Non-Preemptive) Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

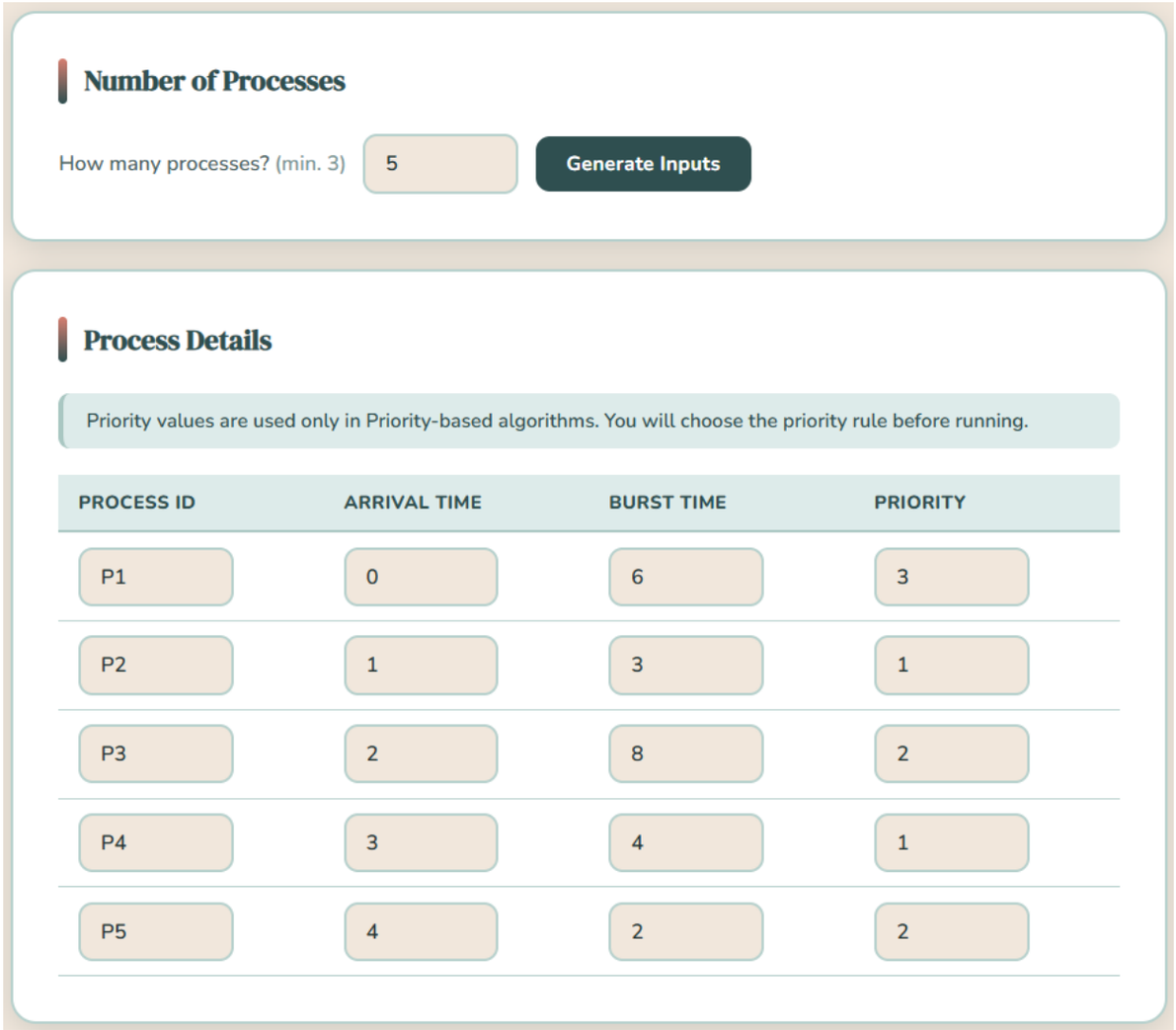

Figure 10.1: *Priority Scheduling (Non-Preemptive) Input Parameters (Test Case 2)*

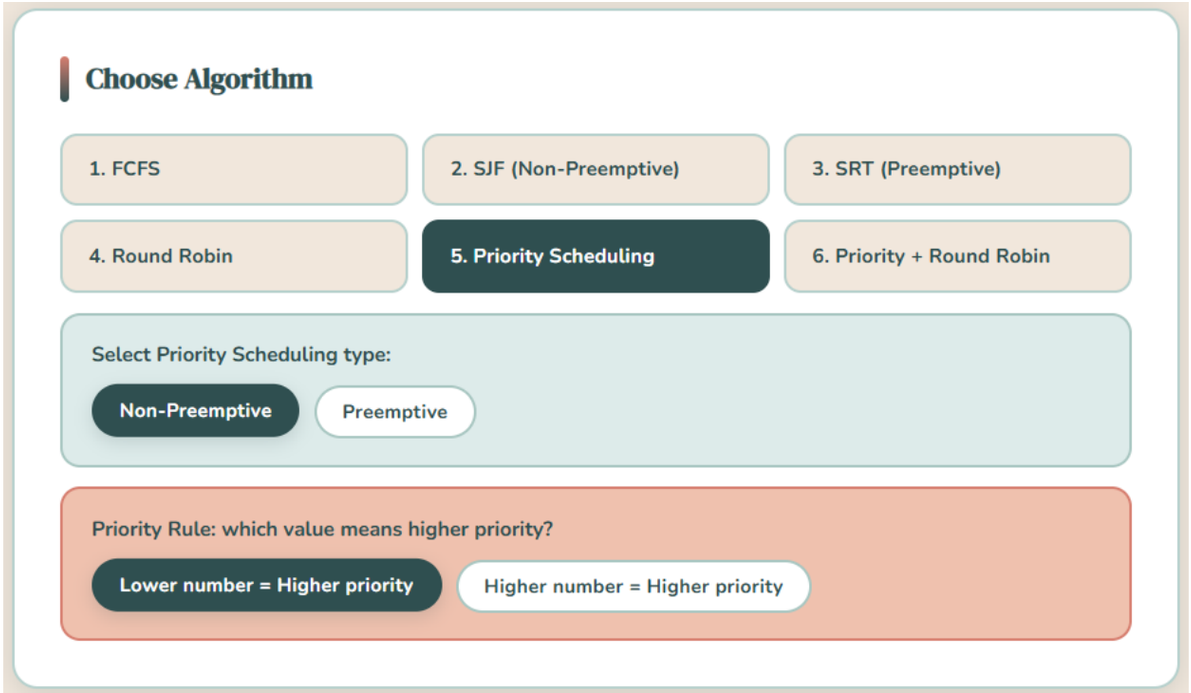

Figure 10.2: *Priority Scheduling (Non-Preemptive) Algorithm Selection (Test Case 2)*

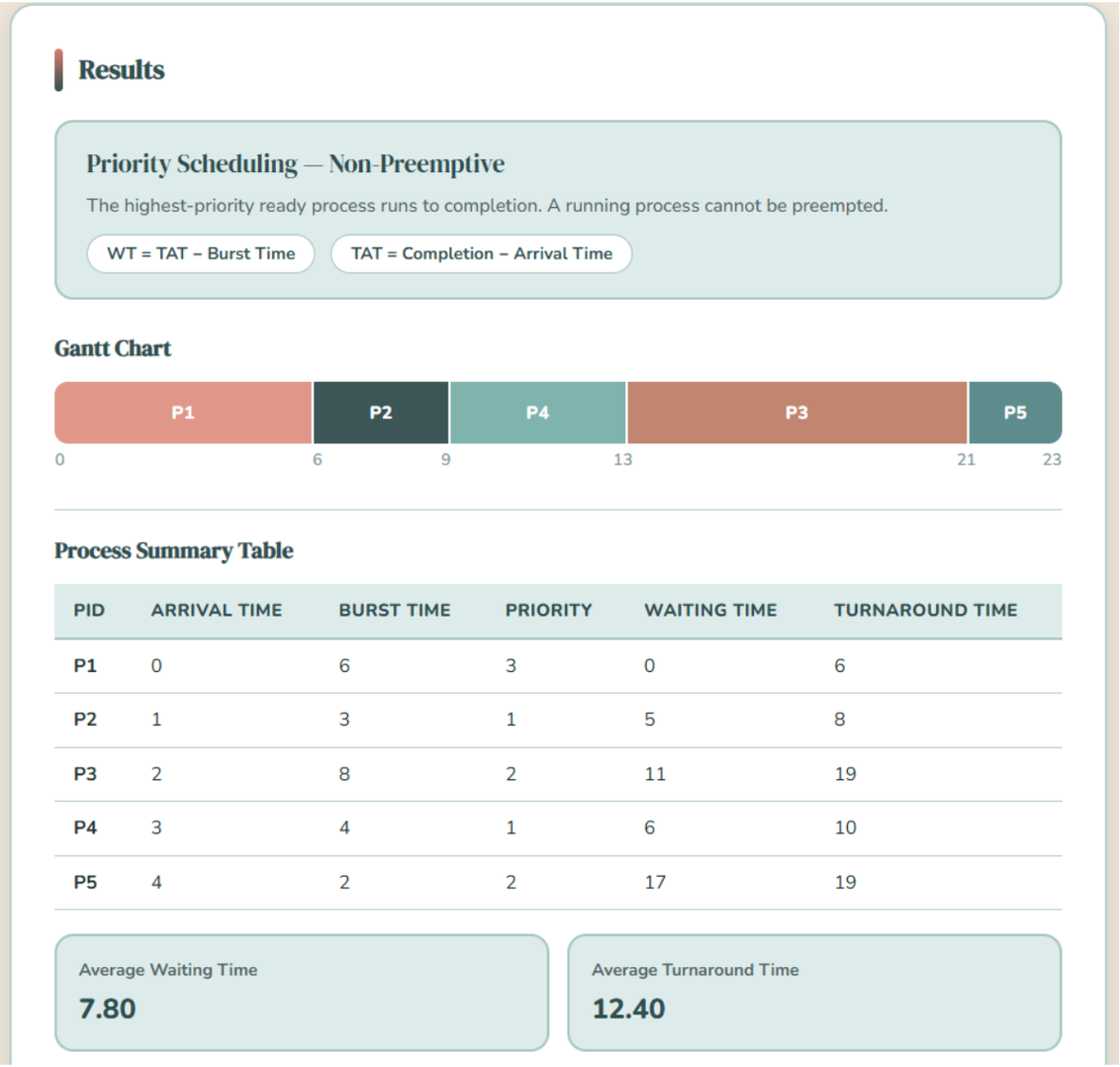

Figure 10.3: *PS (Non-Preemptive) Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.6 Priority Scheduling (Preemptive)

Priority Scheduling (Preemptive) enables the CPU to be preempted each time a higher priority process is preempted. Under this method, when a new process enters the ready queue with a higher priority than the currently running process, the system will immediately interrupt the currently running process and give the CPU to the higher-priority process. This makes it appropriate to real-time and interactive systems as it ensures that critical tasks are done as fast as possible. Yet, this often preemption raises the context switch count leading to greater overhead. As with the non-preemptive variant, the starvation can still happen to the processes of lower priority (Mathur, 2024; Waraich, 2024).

Type: Preemptive

Advantages:

- High responsiveness for critical tasks
- Applicable to real-time systems.
- Ensures priority-based execution

Disadvantages:

- High context switching overhead
- More complex to implement
- Starvation of low-priority processes

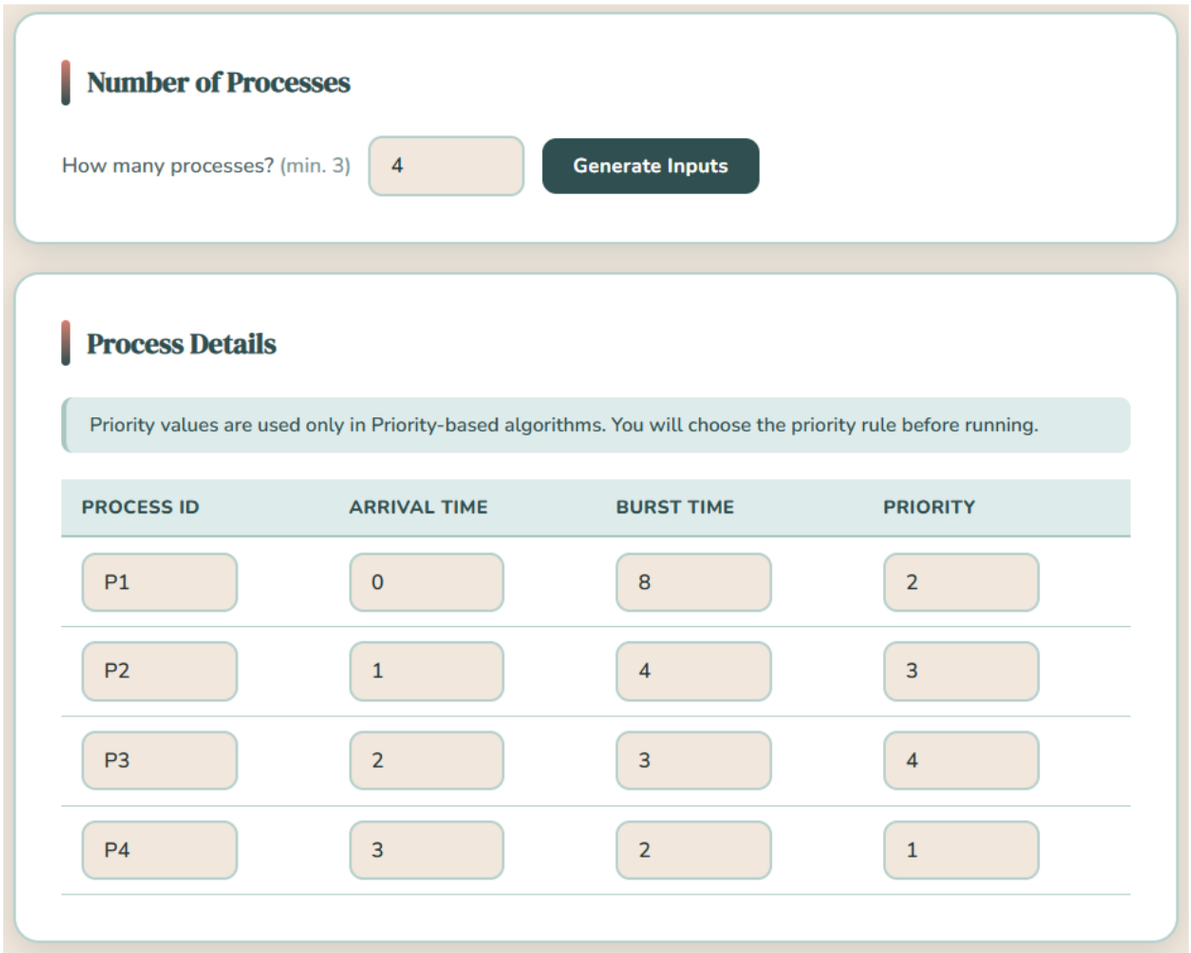

Figure 11.1: *Priority Scheduling (Preemptive) Input Parameters (Test Case 1)*

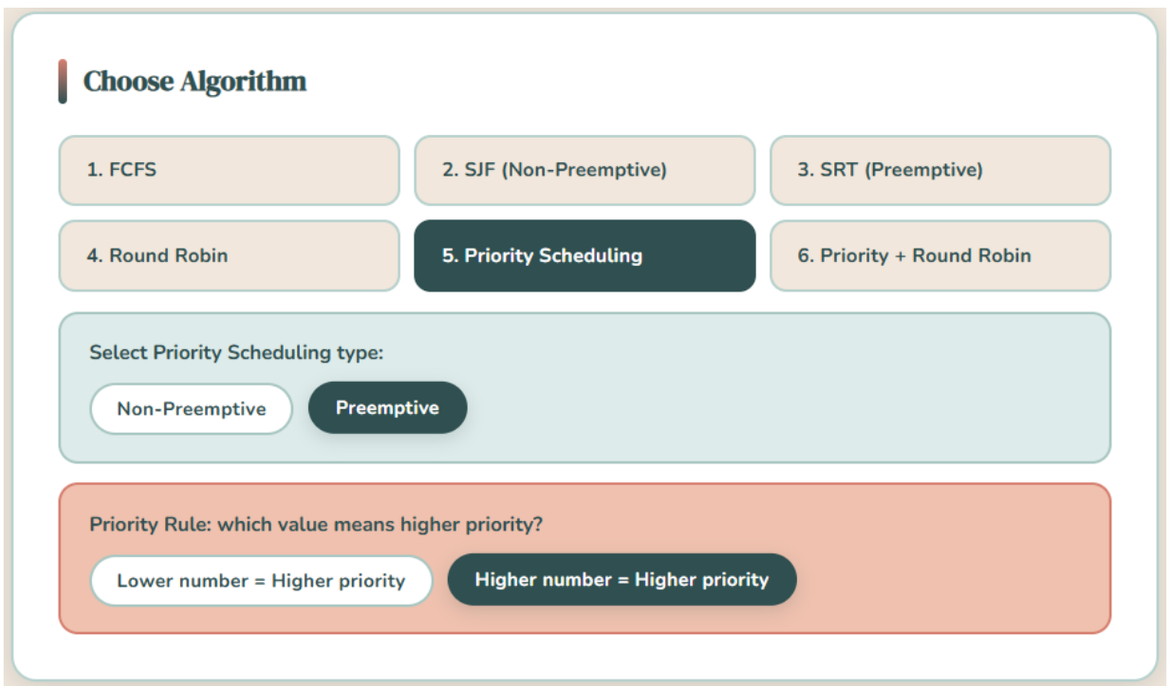

Figure 11.2: *Priority Scheduling (Preemptive) Algorithm Selection (Test Case 1)*

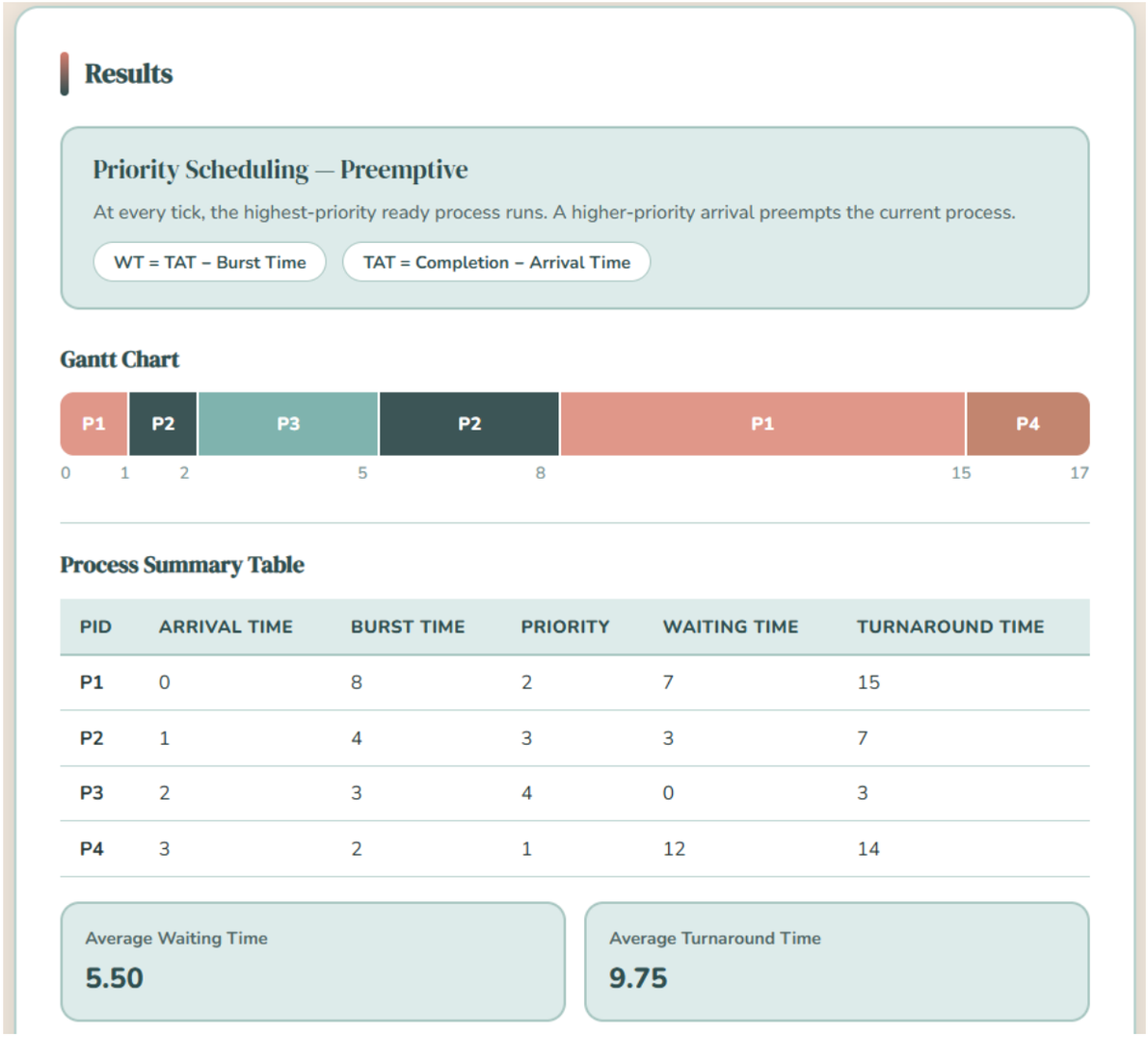

Figure 11.3: *PS (Preemptive) Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 1)*

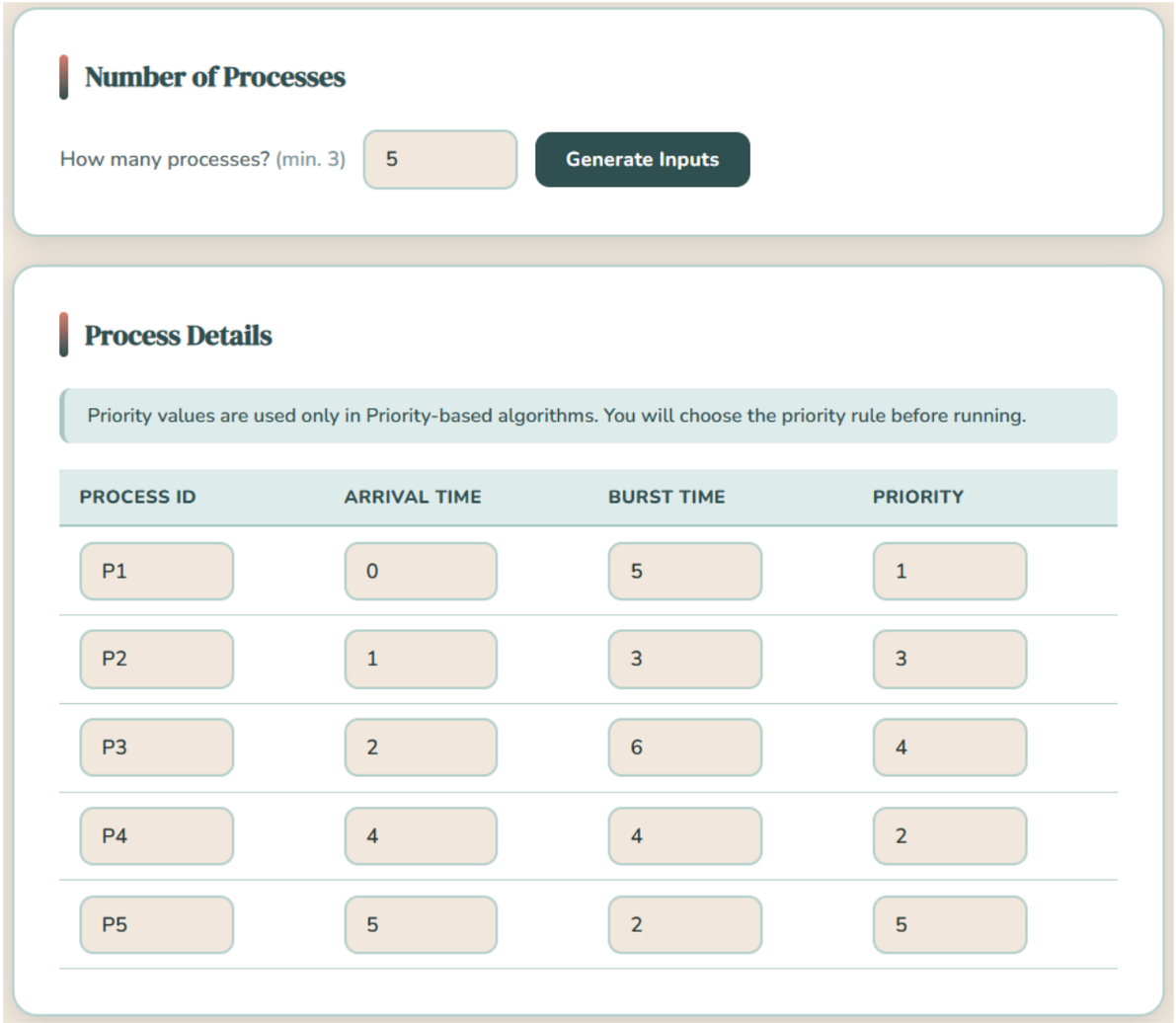

Figure 12.1: *Priority Scheduling (Preemptive) Input Parameters (Test Case 2)*

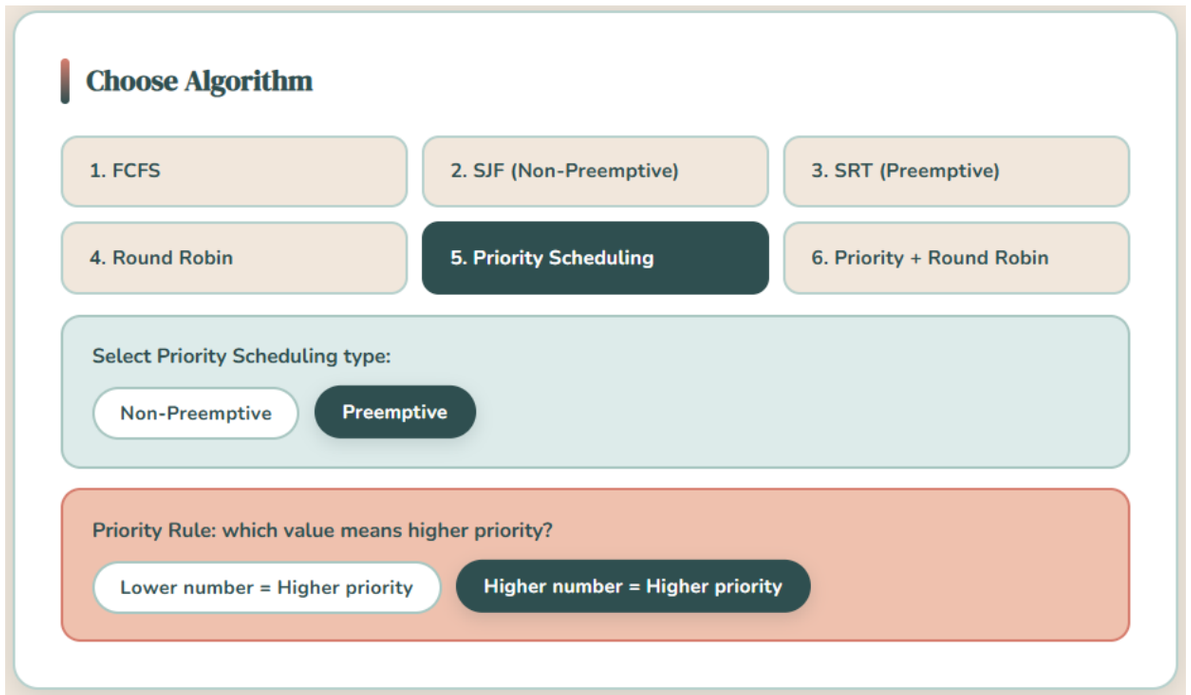

Figure 12.2: *Priority Scheduling (Preemptive) Algorithm Selection (Test Case 2)*

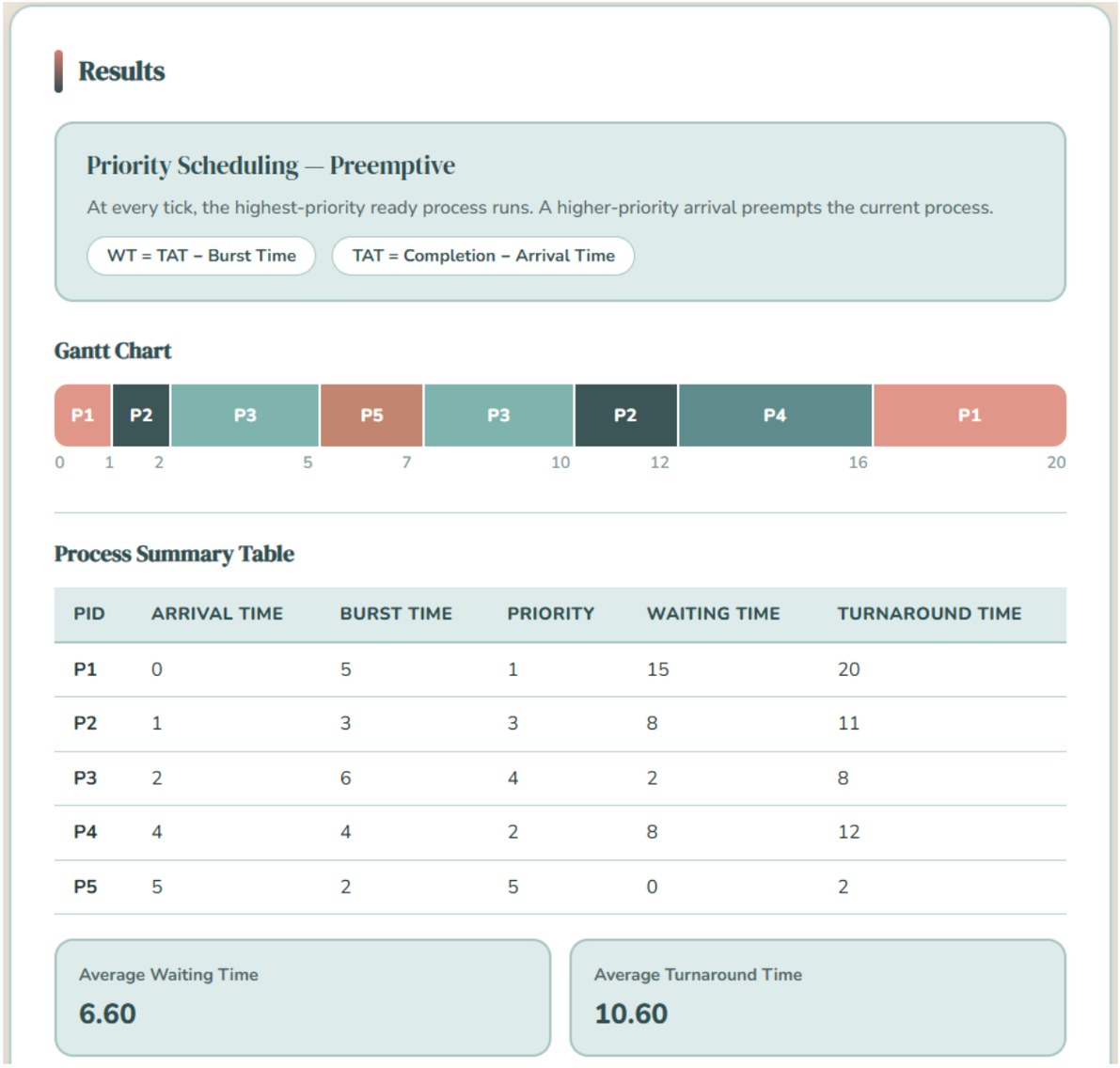

Figure 12.3: *PS (Preemptive) Output (Gantt Chart, Waiting & Turnaround Times, Averages – Test Case 2)*

2.7 Priority Scheduling with Round Robin

Priority Scheduling with Round Robin This is a hybrid scheduling algorithm that combines the benefits of priority-based scheduling and time-sharing scheduling. This approach involves the first grouping of processes according to their degree of priority with the high priority processes being implemented first before low priority processes. In the same level of priority, the Round Robin technique is used where the processes equally share the CPU time with a constant time quantum. This method enhances equity but at the same time key activities are implemented initially. It also helps to reduce starvation in comparison to pure priority scheduling, though not completely. The algorithm is however more complicated to implement and needs to select the time quantum wisely to balance the performance and overhead (Yadav, 2025; Sharma, 2025).

Type: Hybrid (Preemptive)

Advantages:

- Reduces starvation compared to pure priority scheduling
- Maintains fairness within same priority levels
- Balances responsiveness and efficiency

Disadvantages:

- More complex to implement

- Requires proper tuning of time quantum
- Still possible starvation in extreme cases

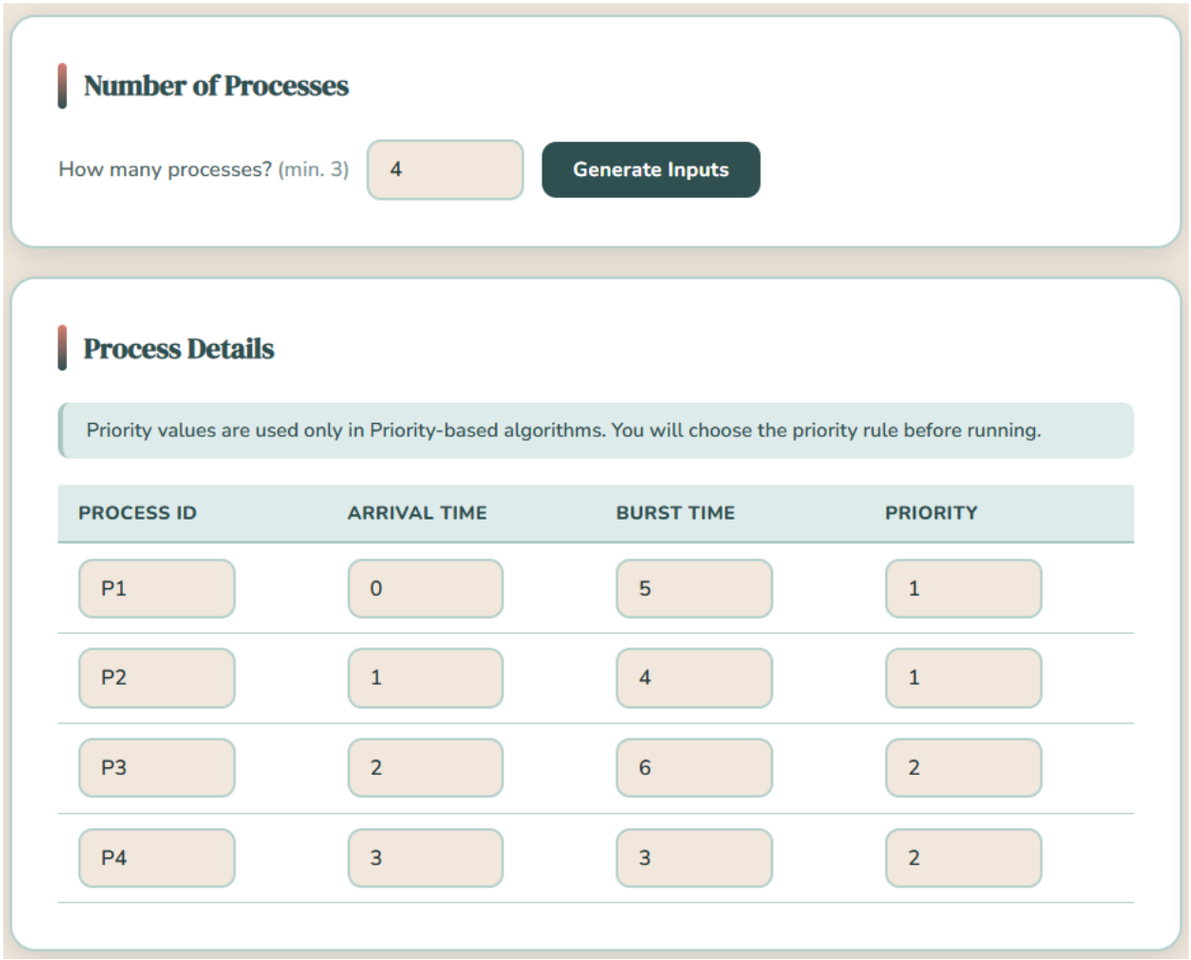

Figure 13.1: *Priority Scheduling with Round Robin Input Parameters (Test Case 1)*

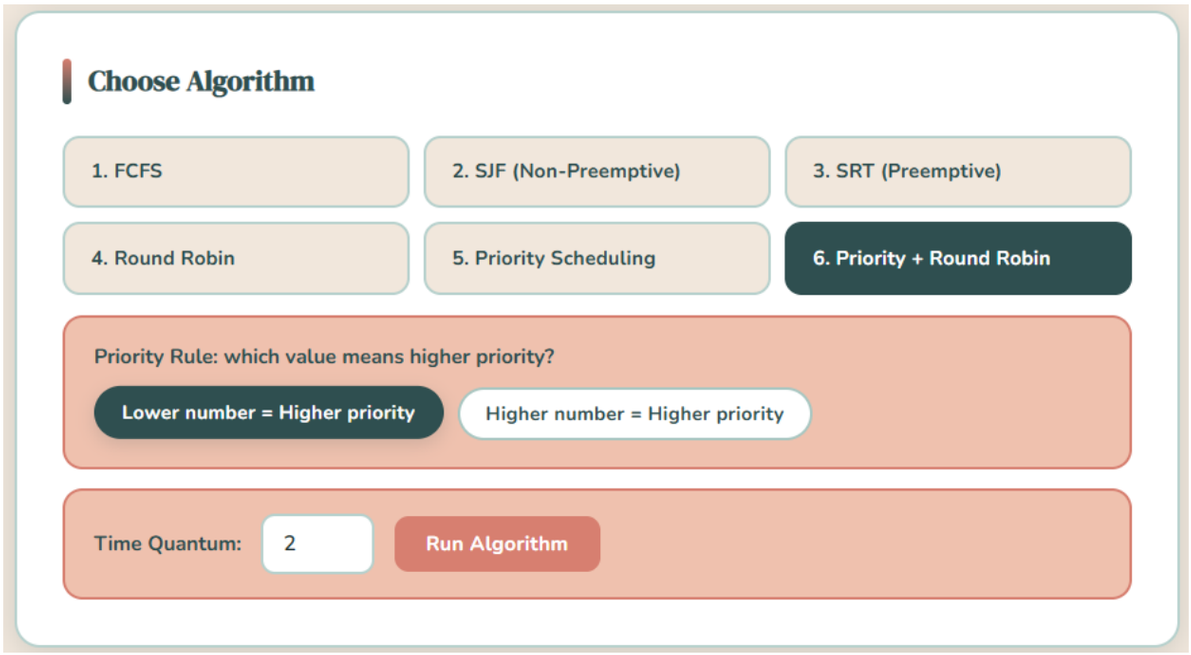

Figure 13.2: *Priority Scheduling with Round Robin Algorithm Selection (Test Case 1)*

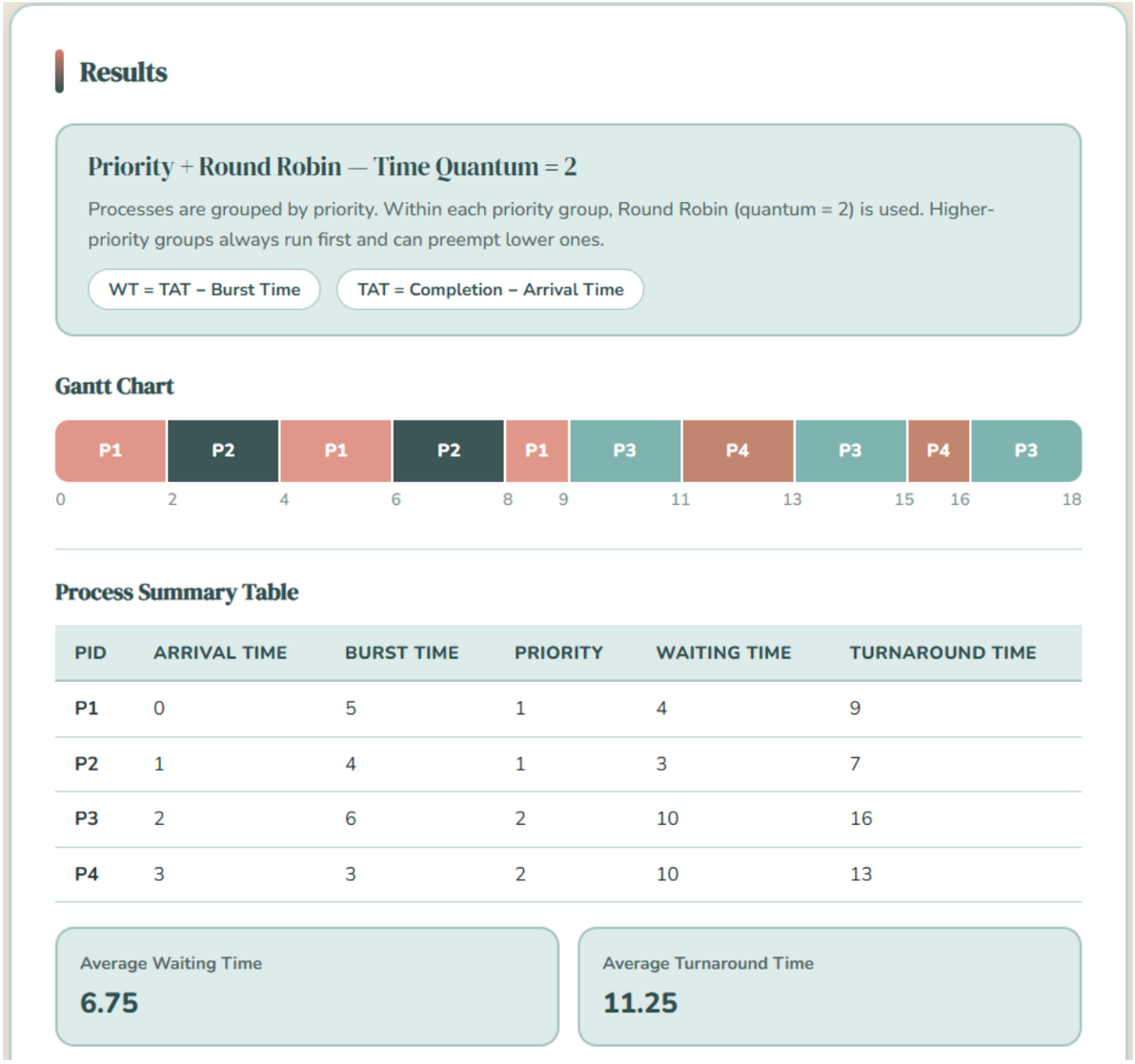

Figure 13.3: *Priority + RR Output (Gantt Chart, WT, TAT, Averages)*

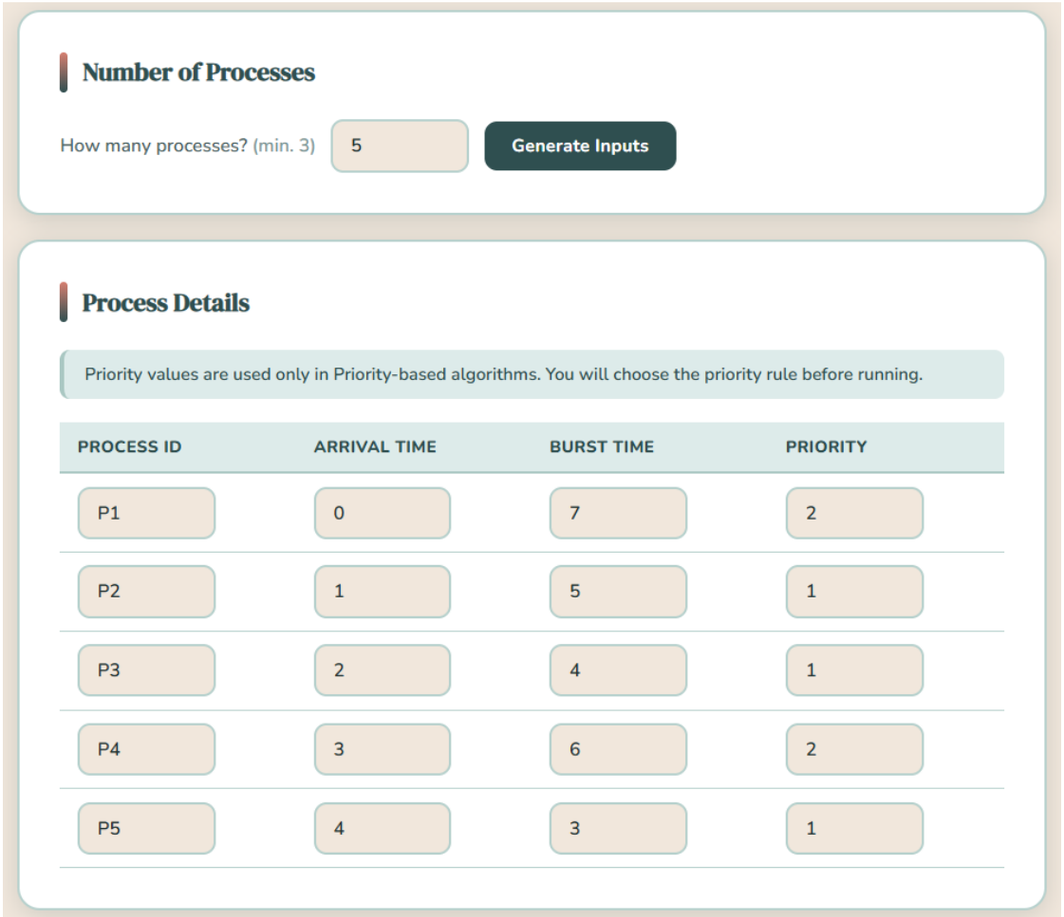

Figure 14.1: *Priority Scheduling with Round Robin Input Parameters (Test Case 2)*

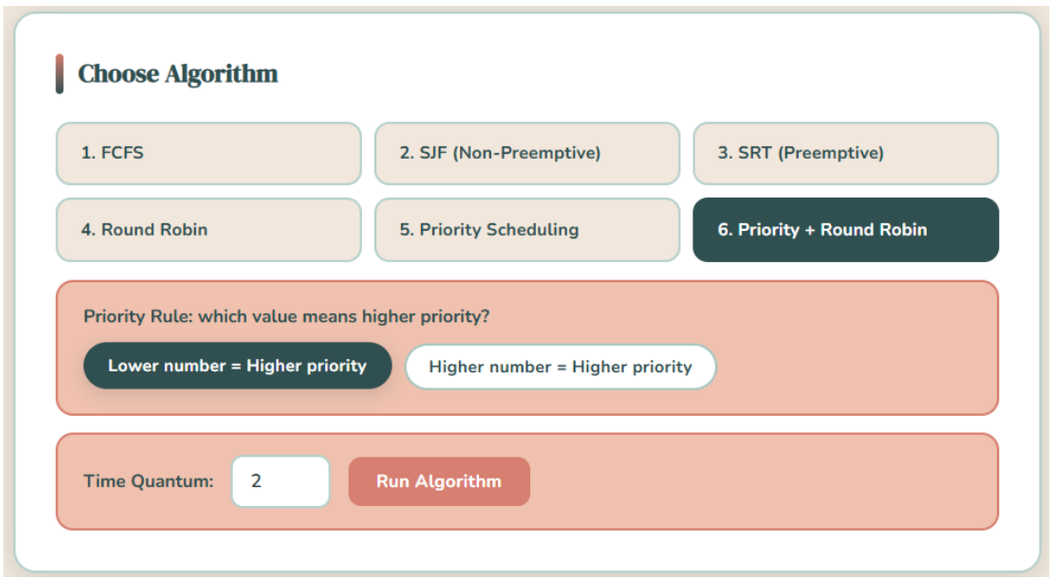

Figure 14.2: *Priority Scheduling with Round Robin Algorithm Selection (Test Case 2)*

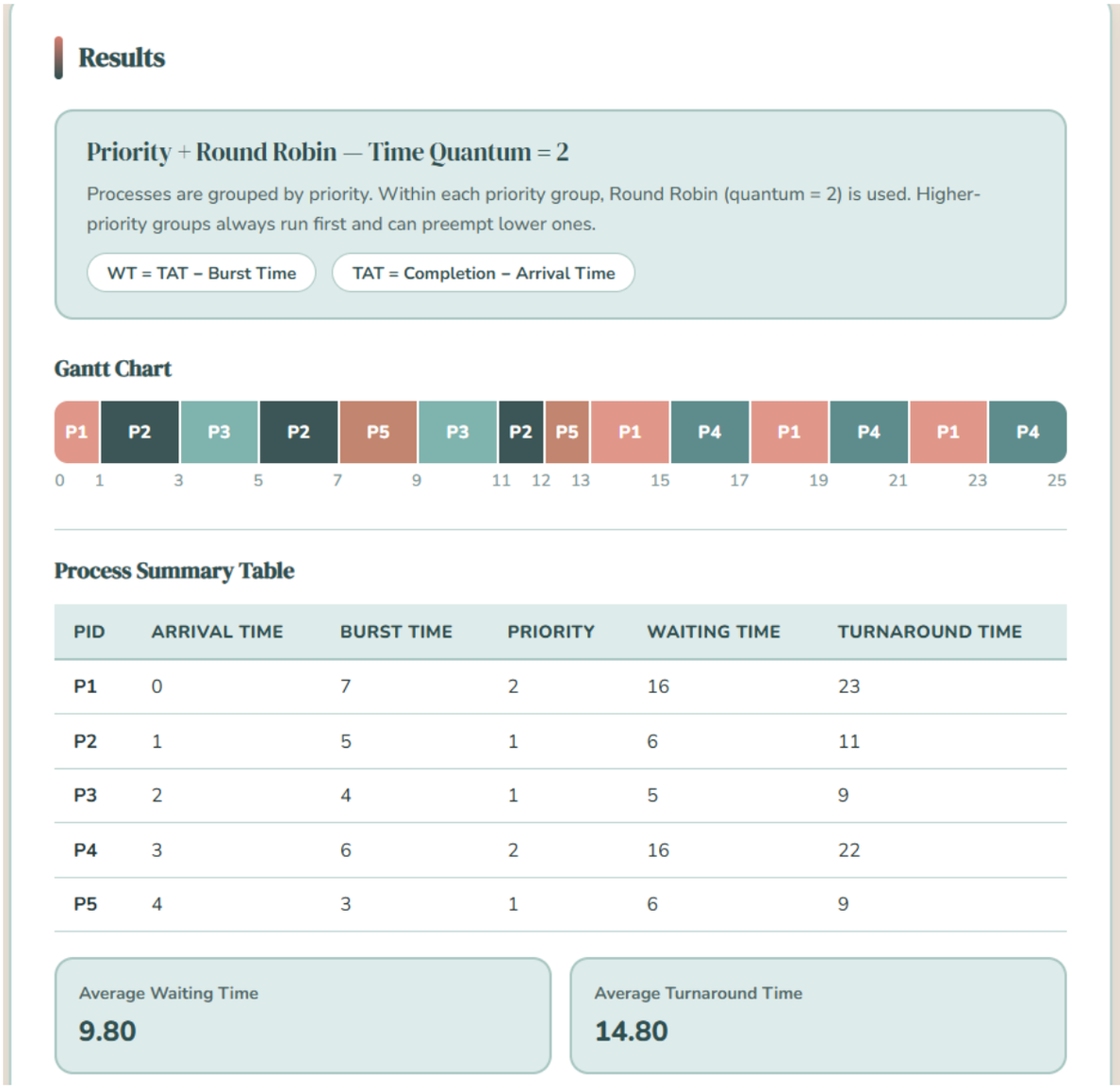

Figure 14.3: *Priority + RR Output (Gantt Chart, WT, TAT, Averages)*

2.8 Comparison of Results Across Algorithms

The following comparison is based on the results obtained from the presented test cases and corresponding screenshots, which include the input parameters, selected algorithms, Gantt charts, and computed Waiting Time (WT) and Turnaround Time (TAT), along with their averages. With the analysis of these outputs, it is more convenient to see how each scheduling algorithm manages the execution of the processes, especially in relation to order, interruptions, and overall performance. The Gantt charts visually represent the execution sequence of each algorithm, and the calculated metrics give quantitative evidence of the performance of each algorithm under the same set of processes.

According to the carried out test cases, the algorithms demonstrated significant variations in the execution behavior and performance metrics. Although theoretically optimal, the results showed that in some cases, depending on the arrival pattern of processes, the average turnaround time of Shortest Job First (SJF) was relatively large. Likewise, Shortest Remaining Time (SRT) enhanced responsiveness via preemption, yet the frequency of interruptions had an impact on the execution of longer processes. First Come First Served (FCFS) adhered to an order of arrival which meant that there was a simple flow of execution and longer waiting times when long processes were early. Round Robin (RR) enhanced fairness as it equally distributed CPU time but it also created extra overhead due to context switching. Priority

Scheduling ensured that high-priority processes were executed faster and causes delays on lower priority, while Priority Scheduling with Round Robin ensured more balanced approached by making priority processes executed fairly. All in all, the findings confirm that the performance of algorithms in any context differs according to the input processes and priorities of the system and no single algorithm is optimal in all scenarios.

III. Summary / Takeaways

3.1 Key Insights Gained

It became clear through the application of various CPU scheduling algorithms, that each algorithm employs a different method in the management of process execution, which directly influences system performance. The study has demonstrated the effect of scheduling decisions on important measures like Waiting Time (WT) and Turnaround Time (TAT) and how these measures can be influenced by the order of process execution and the behavior of algorithms. The development and testing of the simulator helped in better understanding the theoretical concepts through the actual execution and the use of Gantt charts helped visualize the process of scheduling processes over time and to understand the behavior of each algorithm better.

3.2 Observed Differences in Performance

The algorithms adopted exhibited distinct variations in regard to efficiency, fairness, and responsiveness. Algorithms such as SJF and SRT were generally supposed to reduce waiting and turnaround times but might cause delays on longer processes based on the input set. FCFS was easy to understand and follow but tended to increase the waiting time since it strictly adhered to the order of arrival. Round Robin was a better approach to fairness and responsiveness, as it provided equal CPU time to all processes, but introduced additional overhead due to frequent context switching. Priority Scheduling gave preference to the high priority processes, which improved their performance at the expense of the low priority processes, which may have been delayed. Priority with Round Robin The hybrid approach of Priority with Round Robin gave a fairer result and controlled priority as well, at the cost of the low priority processes, which may have been blocked.

3.3 Practical Relevance in Modern Operating Systems

CPU scheduling is an important task in current operating systems, particularly in a setting where several processes compete over the scarce CPU resources. Round robin and other algorithms are widely used in time sharing systems to ensure that the system is responsive but priority based scheduling is necessary in systems where some tasks need to be run more urgently than others. The study illustrates the application of various scheduling strategies based on the requirements of the computing system, and the study explains the significance of applying the right algorithm to achieve efficiency, stability and optimal utilization of resources in real-world computing systems.

3.4 Challenges Encountered and Solutions

In the development of the scheduling simulator, a number of difficulties were experienced, especially in the implementation of preemptive algorithms like SRT and Priority Scheduling, in which the processes are required to be interrupted and resumed correctly. It was also necessary to carefully validate and consistency in formula application in ensuring that the waiting time and turnaround time were correctly computed across all algorithms. Also, it was difficult to choose a proper time quantum of Round Robin, which directly influenced the performance and responsiveness. These difficulties were overcome by debugging step by step, repeated testing with different test cases, and manual checking of the results, which resulted in a workable and reliable implementation of all the scheduling algorithms.

IV. References

Unstop. (2026). CPU scheduling in operating systems | All concepts explained. **https://unstop.com/blog/cpu-scheduling-in-operating-system/amp**

GeeksforGeeks. (2026, April 14). CPU scheduling in operating systems. **https://www.geeksforgeeks.org/operating-systems/cpu-scheduling-in-operating-systems/**

Mathur, T. (2022, April 28). CPU scheduling. Scaler Topics. **https://www.scaler.com/topics/operating-system/cpu-scheduling/**

Awobode, Y. (2022, March 24). First come, first served (FCFS). Webopedia. **https://www.webopedia.com/computers/fcfs/**

GeeksforGeeks. (2025, July 12). Introduction of shortest remaining time first (SRTF) algorithm. **https://www.geeksforgeeks.org/operating-systems/introduction-of-shortest-remaining-time-first-srtf-algorithm/**

GeeksforGeeks. (2026, January 6). Round robin scheduling in operating system. **https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system/**

GeeksforGeeks. (2026, January 7). Shortest job first (SJF) CPU scheduling. **https://www.geeksforgeeks.org/operating-systems/shortest-job-first-or-sjf-cpu-scheduling/**

Mathur, T. (2024, May 2). Priority scheduling algorithm. Scaler Topics. **https://www.scaler.com/topics/operating-system/priority-scheduling-algorithm/**

Sharma, V. (2025, October 10). Round robin scheduling in OS. Scaler Topics. **https://www.scaler.com/topics/round-robin-scheduling-in-os/**

TutorialsPoint. (n.d.). Process scheduling algorithms. **https://www.tutorialspoint.com/operating_system/os_process_scheduling_algorithms.htm**

Waraich, K. (2024, August 6). Non-preemptive priority based scheduling. Naukri Code360. **https://www.naukri.com/code360/library/non-preemptive-priority-based-scheduling**

Yadav, A. (2025, July 31). Scheduling algorithms: FCFS, SJF, RR, priority. NamasteDev. **https://namastedev.com/blog/scheduling-algorithms-fcfs-sjf-rr-priority/**

V. Repository Link

GitHub Repository: https://github.com/MaCarmelaVeronicaVergara/VergaraMaCarmelaVeronicaA-BSCS3B-CPUScheduling

Video Demonstration Link: https://drive.google.com/drive/folders/1y8smWg2f0GmteJ1B-XVunb3_CrQPv5ri?usp=sharing

Video Demonstration. A 5 to 8-minute split-screen recording demonstrating all six CPU scheduling algorithms with a clear voice-over narration. The video shows all program input parameters, Gantt chart outputs, computed Waiting Time (WT) and Turnaround Time (TAT) per process, and the computed averages for each algorithm. The presenter's face is visible during the introduction and conclusion.

- **Source Code.** The complete source code of the CPU Scheduling Simulator, consisting of three files: (a) Vergara_Ma_Carmela_Veronica_BSCS3B_Index.html — main HTML structure and layout; (b) Vergara_Ma_Carmela_Veronica_BSCS3B_Style.css — all visual styling using CSS custom properties; (c) Vergara_Ma_Carmela_Veronica_BSCS3B_Script.js — all scheduling logic, UI behavior, Gantt chart rendering, and results computation. All files include comments explaining core logic and key functions.
- **Executable File.** The program is a web-based application built with standard HTML, CSS, and JavaScript. No installation or compilation is required. The program runs directly in any modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge, entirely offline without any server or internet connection.
- **Instructions on How to Run the Program.**
  Step 1: Download or clone the repository from the link provided above.
  Step 2: Open the downloaded folder.
  Step 3: Double-click or open Vergara_Ma_Carmela_Veronica_BSCS3B_Index.html in any modern web browser.
  Step 4: The CPU Scheduling Simulator will load immediately. No additional software, server, or internet connection is needed. All three source files (HTML, CSS, JS) must remain in the same folder for the program to work correctly.

VI. Source Code

File 1: Vergara_Ma_Carmela_Veronica_BSCS3B_Index.html

This file defines the complete structure and layout of the CPU Scheduling Simulator. It contains four main sections (steps): number of processes input, process details table, algorithm selection panel, and results display. All interactive elements call JavaScript functions defined in the script file.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>CPU Scheduling Simulator</title>

  <!-- Google Fonts: DM Serif Display for headings, Nunito for body text -->
  <link
href="https://fonts.googleapis.com/css2?family=DM+Serif+Display:ital@0;1&famil
y=Nunito:wght@400;500;600;700;800&display=swap" rel="stylesheet" />

  <!-- External stylesheet for all visual styling -->
  <link rel="stylesheet" href="Vergara_Ma_Carmela_Veronica_BSCS3B_Style.css"
/>
</head>
<body>

  <!-- ===== HEADER =====
       Displays the program title and list of supported algorithms.
       Purely decorative / informational — no interactive elements. -->
  <header>
    <h1> CPU Scheduling Algorithms</h1>
    <p>Simulate FCFS · SJF · SRT · Round Robin · Priority · Priority + RR</p>
  </header>

  <!-- ===== PROGRESS STEPS =====
       Shows the user which step they are currently on (1 to 4).
       Each pill is updated by JavaScript (setActivePill function)
       to show "active" (current step) or "done" (completed step). -->
  <div class="progress-bar">
    <div class="step-pill active" id="pill1"><span class="num">1</span> Number
of Processes</div>
    <div class="step-connector"></div>
    <div class="step-pill" id="pill2"><span class="num">2</span> Process
Details</div>
    <div class="step-connector"></div>
    <div class="step-pill" id="pill3"><span class="num">3</span> Choose
Algorithm</div>
    <div class="step-connector"></div>
    <div class="step-pill" id="pill4"><span class="num">4</span> Results</div>
  </div>

  <main>

    <!-- ===== STEP 1: NUMBER OF PROCESSES =====
         The user enters how many processes to simulate (minimum 3).
         Clicking "Generate Inputs" calls generateInputs() in the JS file,
         which builds the input table rows for Step 2. -->
    <section class="card" id="step1">
      <div class="card-header">
        <div class="card-icon"></div>
        <h2>Number of Processes</h2>
      </div>
      <div class="input-row">
        <label for="numProcesses">How many processes? <em style="color:var(--
text-light);font-style:normal;">(min. 3)</em></label>
        <input type="number" id="numProcesses" min="3" value=""
placeholder="e.g. 3" />
        <button onclick="generateInputs()">Generate Inputs</button>
      </div>
    </section>

    <!-- ===== STEP 2: PROCESS DETAILS =====
         A table where the user fills in each process's details:
         - Process ID (e.g. P1, P2)
         - Arrival Time (when the process enters the ready queue)
         - Burst Time (how long the process needs the CPU)
         - Priority (used only by Priority-based algorithms)
         Rows are dynamically added here by generateInputs() in JS. -->
    <section class="card hidden" id="step2">
      <div class="card-header">
        <div class="card-icon"></div>
```

```
        <h2>Process Details</h2>
      </div>

      <!-- Note reminding the user that priority is only used in certain
algorithms -->
      <p class="note"> Priority values are used only in Priority-based
algorithms. You will choose the priority rule before running.</p>

      <div class="table-wrapper">
        <table id="processTable">
          <thead>
            <tr>
              <th>Process ID</th>
              <th>Arrival Time</th>
              <th>Burst Time</th>
              <th>Priority</th>
            </tr>
          </thead>
          <!-- Table rows are inserted here dynamically by JS -->
          <tbody id="processBody"></tbody>
        </table>
      </div>
    </section>

    <!-- ===== STEP 3: CHOOSE ALGORITHM =====
         Buttons for each scheduling algorithm.
         - FCFS, SJF, SRT run directly when clicked.
         - Round Robin and Priority+RR show a Time Quantum input first.
         - Priority Scheduling shows a dropdown to pick Non-Preemptive or
Preemptive.
         All buttons call JS functions defined in the script file. -->
    <section class="card hidden" id="step3">
      <div class="card-header">
        <div class="card-icon"></div>
        <h2>Choose Algorithm</h2>
      </div>

      <!-- Main algorithm selection buttons -->
      <div class="algo-grid">
        <button class="algo-btn" onclick="runAlgo('fcfs', this)">1.
FCFS</button>
        <button class="algo-btn" onclick="runAlgo('sjf', this)">2. SJF (Non-
Preemptive)</button>
        <button class="algo-btn" onclick="runAlgo('srt', this)">3. SRT
(Preemptive)</button>
        <button class="algo-btn" onclick="askQuantum('rr', this)">4. Round
Robin</button>
        <button class="algo-btn" onclick="showPriorityChoice(this)">5.
Priority Scheduling</button>
        <button class="algo-btn" onclick="askQuantum('priority_rr', this)">6.
Priority + Round Robin</button>
      </div>

      <!-- Priority sub-choice panel: appears when "Priority Scheduling" is
clicked.
           Lets the user choose between Non-Preemptive and Preemptive modes. -
->
      <div class="prio-dropdown hidden" id="prioDropdown">
        <label> Select Priority Scheduling type:</label>
        <div class="prio-choices">
          <button class="prio-choice-btn"
onclick="runPrioType('priority_np')">Non-Preemptive</button>
          <button class="prio-choice-btn"
onclick="runPrioType('priority_p')">Preemptive</button>
        </div>
      </div>

      <!-- Priority Rule panel: appears for any Priority-based algorithm.
```

```
          The user selects whether a lower or higher number means higher
priority. -->
      <div class="prio-dropdown hidden" id="prioRuleBox"
style="background:var(--blush);border-color:var(--rose);">
        <label> Priority Rule: which value means higher priority?</label>
        <div class="prio-choices">
          <button class="prio-choice-btn"
id="ruleLow"  onclick="setPrioRule('lower')" >Lower number = Higher
priority</button>
          <button class="prio-choice-btn" id="ruleHigh"
onclick="setPrioRule('higher')">Higher number = Higher priority</button>
        </div>
      </div>

      <!-- Time Quantum panel: appears only for Round Robin and Priority+RR.
           The user enters how many time units each process gets per turn. -->
      <div class="quantum-panel hidden" id="quantumRow">
        <label for="quantum"> Time Quantum:</label>
        <input type="number" id="quantum" min="1" value="2" />
        <button onclick="runWithQuantum()">Run Algorithm</button>
      </div>
    </section>

    <!-- ===== STEP 4: RESULTS =====
         Displays the output after an algorithm runs:
         1. Algorithm name, description, and formulas used
         2. Gantt Chart — visual timeline of process execution
         3. Summary Table — per-process Waiting Time and Turnaround Time
         4. Average boxes — Average WT and Average TAT
         5. Action buttons to re-run or start over
         All content here is filled in dynamically by displayResults() in JS.
-->
    <section class="card hidden" id="step4">
      <div class="card-header">
        <div class="card-icon"></div>
        <h2>Results</h2>
      </div>

      <!-- Algorithm name, short description, and formula pills -->
      <div class="algo-result-header">
        <div class="algo-result-name" id="algoLabel">—</div>
        <div class="algo-result-desc" id="algoDesc">—</div>
        <div class="algo-formula" id="algoFormula"></div>
      </div>

      <!-- Gantt Chart: visual bar showing the order and duration of process
execution -->
      <div class="result-section">
        <h4> Gantt Chart</h4>
        <!-- ganttChart div is filled by JS using flex-based proportional
blocks -->
        <div id="ganttChart" class="gantt-wrapper"></div>
      </div>

      <!-- Horizontal divider line between chart and table -->
      <div class="divider"></div>

      <!-- Process Summary Table: shows WT and TAT per process, sorted by PID
-->
      <div class="result-section">
        <h4> Process Summary Table</h4>
        <div class="table-wrapper">
          <table id="resultTable">
            <thead>
              <tr>
                <th>PID</th>
                <th>Arrival Time</th>
                <th>Burst Time</th>
                <th>Priority</th>
```

```
                <th>Waiting Time</th>
                <th>Turnaround Time</th>
              </tr>
            </thead>
            <!-- Table rows are inserted here by JS after the algorithm runs -
->
            <tbody id="resultBody"></tbody>
          </table>

          <!-- Average boxes: shown below the table, filled by JS -->
          <div class="avg-boxes" id="avgBoxes"></div>
        </div>
      </div>

      <!-- Action buttons at the bottom of the results section:
           "Run Another Algorithm" — stays on same process data, goes back to
Step 3
           "Generate New Processes" — clears everything and starts from Step 1
-->
      <div style="display:flex; gap:10px; flex-wrap:wrap;">
        <button class="reset-btn" onclick="resetAll()">Run Another
Algorithm</button>
        <button class="reset-btn" onclick="goToStep1()">Generate New
Processes</button>
      </div>
    </section>

  </main>

  <!-- ===== FOOTER =====
       Displays the student's name and section. -->
  <footer>
    <p>Ma. Carmela Veronica Vergara &mdash; BSCS-3B</p>
  </footer>

  <!-- External JavaScript file containing all scheduling logic and UI
behavior -->
  <script src="Vergara_Ma_Carmela_Veronica_BSCS3B_Script.js"></script>
</body>
</html>
```

File 2: Vergara_Ma_Carmela_Veronica_BSCS3B_Style.css

This file controls all visual appearance: colors, typography, layout, spacing, animations, and component styles. CSS custom properties (variables) are defined in :root for easy theme management.

```
/* ============================================================
   This file controls all the visual appearance of the program:
   colors, fonts, layout, spacing, animations, and component styles.
   It uses CSS custom properties (variables) defined in :root
   so colors can be changed easily in one place.
   ============================================================ */

/* ===== COLOR PALETTE (CSS Variables) =====
   All colors used throughout the program are defined here.
   Using variables makes it easy to update the theme in one place. */
:root {
  --cream:    #f1e7dc;
  --blush:    #efc1ae;
  --rose:     #d77f70;
  --mint:     #ddebea;
  --sage:     #a9c7c2;
  --teal:     #2f4f50;
  --teal-mid: #426768;
```

```
  --white:    #ffffff;
  --text:     #263636;
  --text-mid: #4f6767;
  --text-light:#7e9999;
  --border:   #b7d1cd;
  --shadow:   rgba(60,85,86,0.18);


/* ===== RESET =====
   Removes default browser margin, padding, and box-sizing inconsistencies.
   box-sizing: border-box means padding and border are included in element
width. */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }


/* ===== BODY =====
   Sets the global font, background color, and text color for the whole page.
*/
body {
  font-family: 'Nunito', sans-serif;
  background: var(--cream);
  color: var(--text);
  min-height: 100vh;
}
/* ===== HEADER =====
   The top banner with the program title and subtitle.
   Uses a dark teal background with decorative circles via ::before and
::after. */
header {
  background: var(--teal);
  padding: 40px 40px 36px;
  text-align: center;
  position: relative;
  overflow: hidden;
}
/* Decorative circle in the top-right corner of the header */
header::before {
  content: '';
  position: absolute;
  top: -60px; right: -60px;
  width: 220px; height: 220px;
  border-radius: 50%;
  background: rgba(255,255,255,0.05);
}
/* Decorative circle in the bottom-left corner of the header */
header::after {
  content: '';
  position: absolute;
  bottom: -40px; left: -40px;
  width: 160px; height: 160px;
  border-radius: 50%;
  background: rgba(200,219,215,0.08);
}
/* Main title text in the header */
header h1 {
  font-family: 'DM Serif Display', serif;
  font-size: 2.2rem;
  color: var(--blush);
  letter-spacing: 0.5px;
  position: relative;
}
/* Subtitle text below the main title */
header p {
  color: var(--sage);
  margin-top: 8px;
  font-size: 0.95rem;
  font-weight: 500;
  position: relative;
}
```

```css
/* ===== PROGRESS STEPS =====
   The step indicator row (Step 1 → Step 2 → Step 3 → Step 4).
   Uses flexbox to align pills and connectors in a row. */
.progress-bar {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 0;
  padding: 28px 24px 8px;
  max-width: 700px;
  margin: 0 auto;
}
/* Each step bubble (e.g. "1 Number of Processes") */
.step-pill {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 18px;
  border-radius: 50px;
  font-size: 0.82rem;
  font-weight: 700;
  color: var(--text-light);
  background: var(--white);
  border: 2px solid var(--border);
  transition: all 0.3s ease;
  white-space: nowrap;
  letter-spacing: 0.3px;
}
/* The numbered circle inside each step pill */
.step-pill .num {
  width: 20px; height: 20px;
  border-radius: 50%;
  background: var(--border);
  color: var(--text-light);
  font-size: 0.72rem;
  font-weight: 800;
  display: flex; align-items: center; justify-content: center;
  transition: all 0.3s;
}
/* Style for the currently active step */
.step-pill.active {
  background: var(--teal);
  border-color: var(--teal);
  color: var(--white);
  box-shadow: 0 4px 14px var(--shadow);
}
/* The number circle inside the active step */
.step-pill.active .num {
  background: var(--blush);
  color: var(--teal);
}
/* Style for steps that have already been completed */
.step-pill.done {
  background: var(--mint);
  border-color: var(--sage);
  color: var(--teal);
}
/* The number circle inside a completed step */
.step-pill.done .num {
  background: var(--sage);
  color: var(--teal);
}
/* The thin horizontal line connecting two step pills */
.step-connector {
  width: 32px;
  height: 2px;
  background: var(--border);
  flex-shrink: 0;
}
```

```
/* ===== MAIN CONTENT AREA =====
   Centers the content and stacks all the step cards vertically. */
main {
  max-width: 860px;
  margin: 0 auto;
  padding: 16px 20px 40px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}
/* ===== CARD =====
   Each step (1-4) is displayed inside a card.
   Cards have a white background, rounded corners, and a soft shadow.
   The slideIn animation makes them appear smoothly. */
.card {
  background: var(--white);
  border: 2px solid var(--border);
  border-radius: 18px;
  padding: 32px;
  box-shadow: 0 4px 20px var(--shadow);
  animation: slideIn 0.35s ease;
}
/* Animation: card fades in and slides up when it appears */
@keyframes slideIn {
  from { opacity: 0; transform: translateY(14px); }
  to   { opacity: 1; transform: translateY(0); }
}
/* The top row of a card: contains the colored accent bar and the section
title */
.card-header {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 22px;
}
/* Thin vertical colored bar beside the card title (decorative accent) */
.card-icon {
  width: 6px;
  height: 32px;
  border-radius: 6px;
  background: linear-gradient(to bottom, var(--rose), var(--teal));
}
/* Section title inside each card (e.g. "Process Details") */
.card h2 {
  font-family: 'DM Serif Display', serif;
  font-size: 1.25rem;
  color: var(--teal);
}
/* ===== HIDDEN UTILITY CLASS =====
   Elements with this class are completely hidden from view.
   JavaScript adds/removes this class to show or hide sections. */
.hidden { display: none !important; }
/* ===== INPUT ROW =====
   Used in Step 1 to lay out the label, number input, and button side by side.
*/
.input-row {
  display: flex;
  align-items: center;
  gap: 12px;
  flex-wrap: wrap;
}
/* Label text beside an input field */
.input-row label {
  color: var(--text-mid);
  font-size: 0.92rem;
  font-weight: 600;
}
/* ===== INPUT FIELDS =====
   Styles for all number and text input boxes in the program.
```

```
   Includes a focus highlight so the user knows which field is active. */
input[type="number"], input[type="text"] {
  background: var(--cream);
  border: 2px solid var(--border);
  border-radius: 10px;
  color: var(--text);
  padding: 9px 14px;
  font-size: 0.92rem;
  font-family: 'Nunito', sans-serif;
  font-weight: 600;
  width: 110px;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}
/* Highlight border and glow when the user clicks into a field */
input[type="number"]:focus, input[type="text"]:focus {
  border-color: var(--teal);
  box-shadow: 0 0 0 3px rgba(60,85,86,0.12);
}
/* ===== BUTTONS (default) =====
   Base style for all buttons in the program.
   Includes a hover effect (lift + shadow) and a click effect (press down). */
button {
  background: var(--teal);
  color: var(--white);
  border: none;
  border-radius: 10px;
  padding: 10px 22px;
  font-size: 0.88rem;
  font-weight: 700;
  font-family: 'Nunito', sans-serif;
  cursor: pointer;
  transition: background 0.2s, transform 0.15s, box-shadow 0.2s;
  letter-spacing: 0.3px;
}
/* Hover: button lifts slightly and darkens */
button:hover {
  background: var(--teal-mid);
  transform: translateY(-2px);
  box-shadow: 0 6px 16px var(--shadow);
}
/* Click: button returns to original position */
button:active { transform: translateY(0); }
/* ===== NOTE BOX =====
   The info message shown at the top of Step 2.
   Has a colored left border to draw attention. */
.note {
  background: var(--mint);
  border-left: 4px solid var(--sage);
  padding: 10px 16px;
  border-radius: 8px;
  font-size: 0.87rem;
  color: var(--teal);
  margin-bottom: 18px;
  font-weight: 600;
}
/* ===== TABLES =====
   Used for both the process input table (Step 2) and results table (Step 4).
*/

/* Allows horizontal scrolling on small screens */
.table-wrapper { overflow-x: auto; }

/* Full-width table with no border gaps between cells */
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.88rem;
}
```

```css
/* Table column header cells */
th {
  background: var(--mint);
  color: var(--teal);
  padding: 11px 14px;
  text-align: left;
  border-bottom: 2px solid var(--sage);
  font-weight: 800;
  font-size: 0.82rem;
  letter-spacing: 0.4px;
  text-transform: uppercase;
}
/* Regular table data cells */
td {
  padding: 10px 14px;
  border-bottom: 1px solid var(--border);
  color: var(--text);
}
/* Highlight a table row when the user hovers over it */
tbody tr:hover { background: var(--cream); }

/* Input fields inside the process input table (Step 2) */
#processTable td input[type="text"],
#processTable td input[type="number"] {
  width: 100%; max-width: 110px;
}
/* ===== ALGORITHM SELECTION GRID (Step 3) =====
   Lays out the 6 algorithm buttons in a responsive grid.
   Buttons wrap to a new row automatically on smaller screens. */
.algo-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(210px, 1fr));
  gap: 10px;
  margin-bottom: 0;
}

/* Each individual algorithm button */
.algo-btn {
  background: var(--cream);
  color: var(--teal);
  border: 2px solid var(--border);
  border-radius: 12px;
  padding: 14px 18px;
  font-size: 0.87rem;
  font-weight: 700;
  font-family: 'Nunito', sans-serif;
  cursor: pointer;
  text-align: left;
  transition: all 0.2s;
  letter-spacing: 0.2px;
}

/* Hover: algorithm button turns teal and lifts */
.algo-btn:hover {
  background: var(--teal);
  color: var(--white);
  border-color: var(--teal);
  transform: translateY(-3px);
  box-shadow: 0 8px 20px var(--shadow);
}

/* Active: the currently selected algorithm button stays highlighted */
.algo-btn.active-algo {
  background: var(--teal);
  color: var(--white);
  border-color: var(--teal);
}
/* ===== PRIORITY DROPDOWN PANELS =====
   These panels slide in below the algorithm buttons when the user
```

```
   selects Priority Scheduling or any priority-based algorithm.
   One panel asks for the type (Preemptive or Non-Preemptive),
   another asks for the priority rule (lower or higher = better). */
.prio-dropdown {
  background: var(--mint);
  border: 2px solid var(--sage);
  border-radius: 14px;
  padding: 18px 20px;
  margin-top: 14px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  animation: slideIn 0.25s ease;
}

/* Label text inside a priority panel */
.prio-dropdown label {
  font-weight: 700;
  color: var(--teal);
  font-size: 0.9rem;
}

/* Container for the choice buttons inside a priority panel */
.prio-choices {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

/* Individual choice button (e.g. "Non-Preemptive" or "Lower number = Higher
priority") */
.prio-choice-btn {
  background: var(--white);
  color: var(--teal);
  border: 2px solid var(--sage);
  border-radius: 50px;
  padding: 8px 18px;
  font-size: 0.84rem;
  font-weight: 700;
  cursor: pointer;
  font-family: 'Nunito', sans-serif;
  transition: all 0.2s;
}

/* Hover and selected state for priority choice buttons */
.prio-choice-btn:hover, .prio-choice-btn.selected {
  background: var(--teal);
  color: var(--white);
  border-color: var(--teal);
  transform: translateY(-1px);
  box-shadow: 0 4px 12px var(--shadow);
}


/* ===== QUANTUM PANEL =====
   Shown below the algorithm buttons when Round Robin or Priority+RR is
selected.
   The user types in the Time Quantum value before running the algorithm. */
.quantum-panel {
  background: var(--blush);
  border: 2px solid var(--rose);
  border-radius: 14px;
  padding: 16px 20px;
  margin-top: 14px;
  display: flex;
  align-items: center;
  gap: 14px;
  flex-wrap: wrap;
  animation: slideIn 0.25s ease;
```

```css
}

/* Label inside the quantum panel */
.quantum-panel label {
  font-weight: 700;
  color: var(--teal);
  font-size: 0.9rem;
}

/* Quantum number input box */
.quantum-panel input {
  width: 80px;
  background: var(--white);
}

/* "Run Algorithm" button inside the quantum panel */
.quantum-panel button {
  background: var(--rose);
}

/* Hover: quantum run button changes to teal */
.quantum-panel button:hover { background: var(--teal); }


/* ===== GANTT CHART =====
   Displays the execution timeline of processes after running an algorithm.
   Uses CSS flexbox proportions (flex: <duration>) so blocks are always
   proportionally sized and fit perfectly inside the card — no overflow. */

/* Outer wrapper for the gantt chart — takes full card width */
.gantt-wrapper { width: 100%; }

/* Container that holds the two rows: blocks row + times row */
.gantt-chart {
  display: flex;
  flex-direction: column;
  width: 100%;
}

/* The top row of colored process blocks */
.gantt-blocks {
  display: flex;
  width: 100%;
}

/* Each individual colored block in the Gantt chart.
   The width is NOT set here — it is set by JS using style.flex = duration.
   This makes each block proportional to how long that process ran. */
.gantt-block {
  display: flex;
  align-items: center;
  justify-content: center;
  height: 46px;
  font-weight: 800;
  font-size: 0.82rem;
  border-right: 2px solid var(--white);
  padding: 0 4px;
  color: var(--white);
  letter-spacing: 0.3px;
  overflow: hidden;
  white-space: nowrap;
  min-width: 0;
}

/* Rounded left edge for the first block in the chart */
.gantt-block:first-child { border-radius: 10px 0 0 10px; }

/* Rounded right edge and no right border for the last block */
.gantt-block:last-child  { border-radius: 0 10px 10px 0; border-right: none; }
```

```css
/* IDLE blocks (when CPU is waiting for a process to arrive) use the sage
color */
.gantt-block.idle { background: var(--sage); color: var(--teal); }

/* The bottom row of time labels (start times under each block) */
.gantt-times {
  display: flex;
  width: 100%;
  padding-top: 4px;
}
/* Each time label under a Gantt block.
   Uses the same flex value as the block above it, so labels align perfectly.
*/
.gantt-time {
  font-size: 0.76rem;
  color: var(--text-light);
  text-align: left;
  font-weight: 600;
  min-width: 0;
  overflow: visible;
}
/* The final end-time label has no flex — it just appears at the far right */
.gantt-time-end {
  flex: none;
}


/* ===== PROCESS COLORS =====
   10 different colors for the Gantt chart blocks.
   JS assigns color-0 to the first process, color-1 to the second, etc.
   These colors are from the program's palette for visual consistency. */
.color-0 { background: #E09789; }
.color-1 { background: #3C5556; }
.color-2 { background: #7fb3ae; }
.color-3 { background: #c2856f; }
.color-4 { background: #5e8c8d; }
.color-5 { background: #d4a5a0; }
.color-6 { background: #4a7a7b; }
.color-7 { background: #a06b5e; }
.color-8 { background: #89bab5; }
.color-9 { background: #e8b5a0; }


/* ===== RESULT SECTION =====
   Wraps each output section inside Step 4 (Gantt Chart, Summary Table).
   Adds spacing below each section. */
.result-section { margin-bottom: 28px; }

/* Section headings inside the results card (e.g. "Gantt Chart") */
.result-section h4 {
  font-family: 'DM Serif Display', serif;
  font-size: 1.05rem;
  color: var(--teal);
  margin-bottom: 14px;
}
/* ===== ALGORITHM RESULT HEADER =====
   The info box at the top of Step 4 that shows:
   - The name of the selected algorithm (bold serif text)
   - A short description of how it works
   - Formula pills showing the WT and TAT formulas used */
.algo-result-header {
  background: var(--mint);
  border: 2px solid var(--sage);
  border-radius: 14px;
  padding: 18px 22px;
  margin-bottom: 24px;
}
```

```css
/* Algorithm name (e.g. "First-Come, First-Served (FCFS)") */
.algo-result-name {
  font-family: 'DM Serif Display', serif;
  font-size: 1.15rem;
  color: var(--teal);
  font-weight: 400;
  margin-bottom: 8px;
}

/* Short description text below the algorithm name */
.algo-result-desc {
  font-size: 0.87rem;
  color: var(--text-mid);
  line-height: 1.6;
  font-weight: 500;
}

/* Container for the formula pill badges */
.algo-formula {
  margin-top: 10px;
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
}

/* Each formula badge (e.g. "WT = TAT - Burst Time") */
.formula-pill {
  background: var(--white);
  border: 1.5px solid var(--sage);
  border-radius: 50px;
  padding: 5px 14px;
  font-size: 0.8rem;
  color: var(--teal);
  font-weight: 700;
}
/* ===== AVERAGE BOXES =====
   Two boxes shown below the summary table:
   one for Average Waiting Time, one for Average Turnaround Time.
   They sit side by side and stretch to fill the available width. */
.avg-boxes {
  display: flex;
  gap: 14px;
  margin-top: 18px;
  flex-wrap: wrap;
}

/* Each individual average box */
.avg-card {
  flex: 1;
  min-width: 200px;
  background: var(--mint);
  border: 2px solid var(--sage);
  border-radius: 14px;
  padding: 16px;
}

/* Label text inside an average box (e.g. "Average Waiting Time") */
.avg-title {
  font-size: 0.8rem;
  color: var(--text-mid);
  font-weight: 700;
}

/* The computed average number displayed in large bold text */
.avg-value {
  font-size: 1.3rem;
  font-weight: 800;
  color: var(--teal);
  margin-top: 6px;
```

```
}


/* ===== RESET / ACTION BUTTONS =====
   The "Run Another Algorithm" and "Generate New Processes" buttons
   at the bottom of the Results section.
   They use a lighter style (cream background) to look secondary
   compared to the main teal buttons. */
.reset-btn {
  background: var(--cream);
  color: var(--teal);
  border: 2px solid var(--sage);
  margin-top: 8px;
}

/* Hover: fills with sage green */
.reset-btn:hover {
  background: var(--sage);
  color: var(--teal);
}


/* ===== FOOTER =====
   Simple centered text at the very bottom of the page
   showing the student's name. */
footer {
  text-align: center;
  padding: 24px;
  color: var(--text-light);
  font-size: 0.82rem;
  font-weight: 600;
  border-top: 1px solid var(--border);
}


/* ===== DIVIDER =====
   A thin horizontal line used to visually separate
   the Gantt Chart from the Summary Table in the Results section. */
.divider {
  height: 1px;
  background: var(--border);
  margin: 18px 0;
}
```

File 3: Vergara_Ma_Carmela_Veronica_BSCS3B_Script.js

This file contains all scheduling logic, UI control functions, Gantt chart rendering, and results display. The algorithms are implemented as reusable, modular functions. Helper functions (nonPreemptive and tickSchedule) reduce duplication across similar algorithms.

```
// ============================================================
// This file contains all the logic for the simulator:
// - Reading user input from the form
// - Running each scheduling algorithm
// - Building the Gantt Chart and Results Table
// - Controlling which sections are visible on screen
// ============================================================


// ===== GLOBAL VARIABLES =====
// These store shared data used across multiple functions.
let processes    = [];      // Array of process objects entered by the user
let pendingAlgo  = null;    // Stores which quantum-based algorithm is waiting
to run
```

```
let prioRule     = 'lower'; // Priority rule: 'lower' means lower number =
higher priority
let pendingPrioAlgo = null; // Stores the selected priority algorithm type

// ============================================================
// STEP PILLS — setActivePill(n)
// Updates the progress bar at the top of the page.
// Steps before n are marked "done" (green), step n is "active" (teal).
// ============================================================
function setActivePill(n) {
  for (let i = 1; i <= 4; i++) {
    const p = document.getElementById('pill' + i);
    p.classList.remove('active', 'done');
    if (i < n)  p.classList.add('done');
    if (i === n) p.classList.add('active');
  }
}

// ============================================================
// GENERATE INPUTS — generateInputs()
// Called when the user clicks "Generate Inputs" in Step 1.
// Reads the number of processes and creates one table row per process
// in Step 2, then shows Steps 2 and 3.
// ============================================================
function generateInputs() {
  const n = parseInt(document.getElementById('numProcesses').value);
  if (isNaN(n) || n < 3) { alert(' Please enter at least 3 processes.');
return; }

  const tbody = document.getElementById('processBody');
  tbody.innerHTML = ''; // Clear any existing rows before generating new ones

  // Create one input row for each process
  for (let i = 1; i <= n; i++) {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td><input type="text"   id="pid_${i}" value="P${i}" /></td>
      <td><input type="number" id="at_${i}"  value="0"    min="0" /></td>
      <td><input type="number" id="bt_${i}"  value="${i}" min="1" /></td>
      <td><input type="number" id="pr_${i}"  value="${i}" min="0" /></td>
    `;
    tbody.appendChild(row);
  }

  show('step2'); show('step3');
  setActivePill(2);

  // Smoothly scroll the page down so the user sees the new input table
  document.getElementById('step2').scrollIntoView({ behavior: 'smooth' });
}

// ============================================================
// READ PROCESSES — readProcesses()
// Reads and validates all process inputs from the Step 2 table.
// Stores valid process data into the global "processes" array.
// Returns false if any input is invalid (shows an alert).
// ============================================================
function readProcesses() {
  const n = parseInt(document.getElementById('numProcesses').value);
  processes = [];

  for (let i = 1; i <= n; i++) {
    const pid = document.getElementById(`pid_${i}`).value.trim();
    const at  = parseInt(document.getElementById(`at_${i}`).value);
    const bt  = parseInt(document.getElementById(`bt_${i}`).value);
    const pr  = parseInt(document.getElementById(`pr_${i}`).value);

    // Validate each field before accepting it
```

```
    if (!pid || !/^[a-zA-Z]/.test(pid))  { alert(` P${i}: ID must start with a
letter.`); return false; }
    if (!/\d/.test(pid))                  { alert(` P${i}: ID must contain a
number.`);    return false; }
    if (isNaN(at) || at < 0)             { alert(` P${i}: Arrival Time must be
>= 0.`);    return false; }
    if (isNaN(bt) || bt < 1)             { alert(` P${i}: Burst Time must be
>= 1.`);      return false; }
    if (isNaN(pr) || pr < 0)             { alert(` P${i}: Priority must be >=
0.`);        return false; }

    processes.push({ id: pid, arrival: at, burst: bt, priority: pr });
  }

  return true; // All inputs are valid
}


// ============================================================
// CLEAR ALGO BUTTONS — clearAlgoBtns()
// Resets the algorithm button highlights and hides all panels
// (priority dropdown, priority rule box, quantum input).
// Called every time the user picks a new algorithm.
// ============================================================
function clearAlgoBtns() {
  document.querySelectorAll('.algo-btn').forEach(b =>
b.classList.remove('active-algo'));
  hide('prioDropdown');
  hide('prioRuleBox');
  hide('quantumRow');
  pendingAlgo = null;
  pendingPrioAlgo = null;
}


// ============================================================
// RUN ALGO — runAlgo(algo, btn)
// Handles algorithms that run immediately without needing a quantum:
// FCFS, SJF, and SRT.
// Looks up the correct function, name, description, and formulas,
// then calls that function and passes results to displayResults().
// ============================================================
function runAlgo(algo, btn) {
  if (!readProcesses()) return; // Stop if any input is invalid

  clearAlgoBtns();
  if (btn) btn.classList.add('active-algo'); // Highlight the clicked button

  // Map each algorithm key to: [function, display name, description,
...formulas]
  const map = {
    fcfs: [fcfs, 'First-Come, First-Served (FCFS)',           'Processes are
executed in the order they arrive. Simple and fair — no process jumps the
queue.',                          'WT = TAT - Burst Time', 'TAT = Completion -
Arrival Time'],
    sjf:  [sjf,  'Shortest Job First — Non-Preemptive (SJF)', 'Once the CPU is
free, the available process with the shortest burst time runs to
completion.',                              'WT = TAT - Burst Time', 'TAT =
Completion - Arrival Time'],
    srt:  [srt,  'Shortest Remaining Time — Preemptive (SRT)','At every
moment, the process with the least remaining burst time runs. A new arrival
can preempt the current process.', 'WT = TAT - Burst Time', 'TAT = Completion
- Arrival Time'],
  };

  const [fn, name, desc, ...formulas] = map[algo];
  const [result, gantt] = fn(processes);
  displayResults(result, gantt, name, desc, formulas);
}
```

```
// ============================================================
// SHOW PRIORITY CHOICE — showPriorityChoice(btn)
// Called when the user clicks "5. Priority Scheduling".
// Shows the sub-panel asking Non-Preemptive or Preemptive,
// and the priority rule panel, then scrolls to them.
// ============================================================
function showPriorityChoice(btn) {
  if (!readProcesses()) return;

  clearAlgoBtns();
  if (btn) btn.classList.add('active-algo');

  show('prioDropdown'); // Show the Non-Preemptive / Preemptive choice
  show('prioRuleBox');  // Show the priority rule choice (lower/higher)

  // Scroll so the panels are visible after they appear
  setTimeout(() => {
    document.getElementById('prioDropdown').scrollIntoView({ behavior:
'smooth', block: 'center' });
  }, 100);
}


// ============================================================
// RUN PRIORITY TYPE — runPrioType(algo)
// Called when the user picks "Non-Preemptive" or "Preemptive"
// from the Priority Scheduling sub-panel.
// Runs the correct priority algorithm and shows the results.
// ============================================================
function runPrioType(algo) {
  pendingPrioAlgo = algo;

  // Map priority algorithm keys to their function, name, description, and
formulas
  const map = {
    priority_np: [priorityNP, 'Priority Scheduling — Non-Preemptive', 'The
highest-priority ready process runs to completion. A running process cannot be
preempted.', 'WT = TAT - Burst Time', 'TAT = Completion - Arrival Time'],
    priority_p:  [priorityP,  'Priority Scheduling — Preemptive',     'At
every tick, the highest-priority ready process runs. A higher-priority arrival
preempts the current process.', 'WT = TAT - Burst Time', 'TAT = Completion -
Arrival Time'],
  };

  const [fn, name, desc, ...formulas] = map[algo];
  const [result, gantt] = fn(processes);
  displayResults(result, gantt, name, desc, formulas);
}


// ============================================================
// SET PRIORITY RULE — setPrioRule(rule)
// Called when the user clicks "Lower number" or "Higher number"
// in the priority rule panel.
// Updates the global prioRule variable and highlights the selected button.
// ============================================================
function setPrioRule(rule) {
  prioRule = rule;
  document.getElementById('ruleLow').classList.toggle('selected',  rule ===
'lower');
  document.getElementById('ruleHigh').classList.toggle('selected', rule ===
'higher');
}

// Set "lower" as the default priority rule when the page first loads
setPrioRule('lower');
```

```
// ============================================================
// ASK QUANTUM — askQuantum(algo, btn)
// Called when the user clicks "Round Robin" or "Priority + Round Robin".
// Shows the Time Quantum input panel (and the priority rule panel if needed).
// The algorithm waits until the user clicks "Run Algorithm".
// ============================================================
function askQuantum(algo, btn) {
  if (!readProcesses()) return;

  clearAlgoBtns();
  if (btn) btn.classList.add('active-algo');

  pendingAlgo = algo; // Remember which algorithm to run after the user enters the quantum

  // Priority+RR also needs the priority rule, so show that panel too
  if (algo === 'priority_rr') show('prioRuleBox');

  show('quantumRow'); // Show the time quantum input
}


// ============================================================
// RUN WITH QUANTUM — runWithQuantum()
// Called when the user clicks "Run Algorithm" in the quantum panel.
// Reads the time quantum value, runs the appropriate algorithm,
// and passes the results to displayResults().
// ============================================================
function runWithQuantum() {
  const q = parseInt(document.getElementById('quantum').value);
  if (isNaN(q) || q < 1) { alert(' Time Quantum must be at least 1.'); return; }

  let result, gantt, name, desc, formulas;

  if (pendingAlgo === 'rr') {
    // Round Robin: each process runs for at most q time units per turn
    [result, gantt] = roundRobin(processes, q);
    name     = `Round Robin (RR) — Time Quantum = ${q}`;
    desc     = `Each process gets a fixed time slice (quantum = ${q}). If not finished, it goes to the back of the queue and waits its turn again.`;
    formulas = ['WT = TAT - Burst Time', 'TAT = Completion - Arrival Time'];
  } else {
    // Priority + Round Robin: processes are grouped by priority, RR applied within each group
    [result, gantt] = priorityRR(processes, q);
    name     = `Priority + Round Robin — Time Quantum = ${q}`;
    desc     = `Processes are grouped by priority. Within each priority group, Round Robin (quantum = ${q}) is used. Higher-priority groups always run first and can preempt lower ones.`;
    formulas = ['WT = TAT - Burst Time', 'TAT = Completion - Arrival Time'];
  }

  displayResults(result, gantt, name, desc, formulas);
}


// ============================================================
// HELPER A: Non-Preemptive Scheduler — nonPreemptive(procs_in, selector)
// A reusable template used by SJF and Priority-NP.
// At each step, waits for the CPU to be free, then picks the best
// available process using the "selector" function.
// Once a process starts, it runs fully to completion (no interruption).
// ============================================================
function nonPreemptive(procs_in, selector) {
  const procs = procs_in.map(p => ({ ...p })); // Copy to avoid modifying the original data
  const done = [], gantt = [];
  let time = 0;
```

```
  while (done.length < procs.length) {
    // Get all processes that have arrived and are not yet finished
    const available = procs.filter(p => p.arrival <= time &&
!done.includes(p));

    if (!available.length) {
      // No process is ready — CPU is idle. Jump to the next arrival time.
      const next = Math.min(...procs.filter(p => !done.includes(p)).map(p =>
p.arrival));
      gantt.push({ id: 'IDLE', start: time, end: next });
      time = next;
      continue;
    }

    // Use the selector to pick the best process (e.g. shortest burst, highest
priority)
    const cur    = selector(available);
    const finish = time + cur.burst;

    // Calculate Waiting Time and Turnaround Time for this process
    cur.wt  = time - cur.arrival;    // WT = time process waited before
starting
    cur.tat = finish - cur.arrival;  // TAT = total time from arrival to
completion

    done.push(cur);
    gantt.push({ id: cur.id, start: time, end: finish });
    time = finish;
  }

  return [done, gantt];
}


// ============================================================
// HELPER B: Preemptive Tick-by-Tick Scheduler — tickSchedule(procs_in,
selector)
// A reusable template used by SRT and Priority-P.
// At every single time unit (tick), it re-evaluates which process should run.
// A currently running process can be interrupted (preempted) at any tick.
// ============================================================
function tickSchedule(procs_in, selector) {
  const procs = procs_in.map(p => ({ ...p, remaining: p.burst })); // Add
remaining time tracker
  let time = 0, completed = 0;
  let currentPid = null, startTime = 0; // Track current block for Gantt chart
  const gantt = [];

  while (completed < procs.length) {
    // Get all processes that have arrived and still have remaining burst time
    const available = procs.filter(p => p.arrival <= time && p.remaining > 0);

    if (!available.length) {
      // CPU is idle — record IDLE block and move forward one tick
      if (currentPid !== 'IDLE') {
        if (currentPid) gantt.push({ id: currentPid, start: startTime, end:
time });
        currentPid = 'IDLE'; startTime = time;
      }
      time++;
      continue;
    }

    // Select the best process for this tick using the selector function
    const cur = selector(available);

    // If the running process changed, close the current Gantt block and start
a new one
```

```
    if (currentPid !== cur.id) {
      if (currentPid) gantt.push({ id: currentPid, start: startTime, end: time
});
      currentPid = cur.id;
      startTime  = time;
    }

    cur.remaining--; // Process runs for one tick
    time++;

    // If the process just finished, record its completion time
    if (cur.remaining === 0) {
      cur.completion = time;
      completed++;
    }
  }

  // Close the final Gantt block
  if (currentPid) gantt.push({ id: currentPid, start: startTime, end: time });

  // Calculate WT and TAT from each process's completion time
  const result = procs.map(p => ({
    ...p,
    tat: p.completion - p.arrival,
    wt:  p.completion - p.arrival - p.burst
  }));

  return [result, gantt];
}


// ============================================================
// ALGORITHM 1: FCFS — First-Come, First-Served
// Processes are sorted by arrival time and run in that order.
// No preemption — each process runs fully before the next one starts.
// ============================================================
function fcfs(procs_in) {
  const procs = procs_in.map(p => ({ ...p })).sort((a, b) => a.arrival -
b.arrival);
  let time = 0;
  const gantt = [], result = [];

  for (const p of procs) {
    // If the CPU is free but no process has arrived yet, add an IDLE block
    if (time < p.arrival) {
      gantt.push({ id: 'IDLE', start: time, end: p.arrival });
      time = p.arrival;
    }

    const finish = time + p.burst;

    result.push({ ...p, wt: time - p.arrival, tat: finish - p.arrival });
    gantt.push({ id: p.id, start: time, end: finish });

    time = finish;
  }

  return [result, gantt];
}


// ============================================================
// ALGORITHM 2: SJF — Shortest Job First (Non-Preemptive)
// When the CPU is free, the available process with the shortest
// burst time is selected. Ties are broken by arrival time (FCFS).
// Once selected, it runs to completion without interruption.
// ============================================================
function sjf(procs_in) {
  return nonPreemptive(procs_in, avail => avail.reduce((a, b) => {
```

```
    if (a.burst !== b.burst) return a.burst < b.burst ? a : b;
    return a.arrival <= b.arrival ? a : b; // Tie-breaker: earlier arrival
wins
  }));
}


// ============================================================
// ALGORITHM 3: SRT — Shortest Remaining Time (Preemptive)
// At every tick, the process with the least remaining burst time runs.
// If a new process arrives with less remaining time, it preempts
// the currently running process.
// ============================================================
function srt(procs_in) {
  return tickSchedule(procs_in, avail => avail.reduce((a, b) => {
    if (a.remaining !== b.remaining) return a.remaining < b.remaining ? a : b;
    return a.arrival <= b.arrival ? a : b; // Tie-breaker: earlier arrival
wins
  }));
}


// ============================================================
// ALGORITHM 4: Round Robin (RR)
// Each process gets a fixed time slice called the Time Quantum.
// If it doesn't finish within the quantum, it goes to the back
// of the queue and waits for its next turn.
// ============================================================
function roundRobin(procs_in, quantum) {
  const procs = procs_in.map(p => ({ ...p, remaining: p.burst })).sort((a, b)
=> a.arrival - b.arrival);
  let time = 0, completed = 0, i = 0;
  const queue = [], gantt = [];

  while (completed < procs.length) {
    // Add all processes that have now arrived into the ready queue
    while (i < procs.length && procs[i].arrival <= time)
queue.push(procs[i++]);

    if (!queue.length) {
      // No process ready — CPU is idle, jump to next arrival
      gantt.push({ id: 'IDLE', start: time, end: procs[i].arrival });
      time = procs[i].arrival;
      continue;
    }

    const cur  = queue.shift(); // Take the first process from the front of
the queue
    const start = time;
    const exec  = Math.min(quantum, cur.remaining); // Run for quantum or
remaining time, whichever is less

    time += exec;
    cur.remaining -= exec;
    gantt.push({ id: cur.id, start, end: time });

    // Enqueue any processes that arrived while this process was running
    while (i < procs.length && procs[i].arrival <= time)
queue.push(procs[i++]);

    if (cur.remaining > 0) {
      queue.push(cur); // Not finished — go back to the end of the queue
    } else {
      cur.completion = time; // Finished — record completion time
      completed++;
    }
  }

  const result = procs.map(p => ({
```

```
    ...p,
    tat: p.completion - p.arrival,
    wt:  p.completion - p.arrival - p.burst
  }));

  return [result, gantt];
}


// ============================================================
// ALGORITHM 5: Priority Scheduling (Non-Preemptive)
// When the CPU is free, the available process with the highest priority
// is selected (based on prioRule: lower or higher number wins).
// Ties are broken by arrival time, then by Process ID.
// Once selected, it runs to completion.
// ============================================================
function priorityNP(procs_in) {
  return nonPreemptive(procs_in, avail =>
    avail.sort((a, b) => {
      // Sort by priority first (direction depends on prioRule)
      if (a.priority !== b.priority)
        return prioRule === 'lower' ? a.priority - b.priority : b.priority -
a.priority;
      // Tie-break by arrival time, then alphabetically by ID
      if (a.arrival !== b.arrival) return a.arrival - b.arrival;
      return a.id.localeCompare(b.id);
    })[0] // Pick the first (best) process after sorting
  );
}


// ============================================================
// ALGORITHM 6: Priority Scheduling (Preemptive)
// At every tick, the highest-priority available process runs.
// If a higher-priority process arrives, it immediately preempts
// the current one.
// ============================================================
function priorityP(procs_in) {
  return tickSchedule(procs_in, avail =>
    avail.sort((a, b) => {
      // Sort by priority first (direction depends on prioRule)
      if (a.priority !== b.priority)
        return prioRule === 'lower' ? a.priority - b.priority : b.priority -
a.priority;
      // Tie-break by arrival time, then alphabetically by ID
      if (a.arrival !== b.arrival) return a.arrival - b.arrival;
      return a.id.localeCompare(b.id);
    })[0] // Pick the first (best) process after sorting
  );
}


// ============================================================
// ALGORITHM 7: Priority + Round Robin (Hybrid)
// Processes are placed into separate queues based on their priority level.
// The highest-priority queue always runs first.
// Within each priority queue, Round Robin (time quantum) is used.
// If a higher-priority process arrives mid-execution, it preempts.
// ============================================================
function priorityRR(procs_in, quantum) {
  const procs = procs_in.map(p => ({ ...p, remaining: p.burst })).sort((a, b)
=> a.arrival - b.arrival);
  let time = 0, completed = 0, i = 0;
  const gantt  = [];
  const queues = {}; // Separate ready queue per priority level: { 0: [...],
1: [...], ... }

  while (completed < procs.length) {
    // Enqueue newly arrived processes into their priority group
```

```
    while (i < procs.length && procs[i].arrival <= time) {
      const pr = procs[i].priority;
      if (!queues[pr]) queues[pr] = [];
      queues[pr].push(procs[i++]);
    }

    // Check which priority levels have waiting processes
    const active = Object.keys(queues).filter(pr => queues[pr].length >
0).map(Number);

    if (!active.length) {
      // No process ready — CPU is idle
      gantt.push({ id: 'IDLE', start: time, end: procs[i].arrival });
      time = procs[i].arrival;
      continue;
    }

    // Select the best priority level based on the chosen rule
    const curPr = prioRule === 'lower' ? Math.min(...active) :
Math.max(...active);
    const cur   = queues[curPr].shift(); // Take first process from that
priority group
    const start = time;
    let done    = false;

    // Run tick-by-tick for up to quantum ticks, checking for preemption each
tick
    for (let t = 0; t < quantum; t++) {
      time++;
      cur.remaining--;

      // Add any processes that arrived during this tick
      while (i < procs.length && procs[i].arrival <= time) {
        const pr = procs[i].priority;
        if (!queues[pr]) queues[pr] = [];
        queues[pr].push(procs[i++]);
      }

      // Check if the process just finished
      if (cur.remaining === 0) {
        cur.completion = time;
        completed++;
        gantt.push({ id: cur.id, start, end: time });
        done = true;
        break;
      }

      // Check if a higher-priority process has arrived — if so, preempt
      const allActive = Object.keys(queues).filter(pr => queues[pr].length >
0).map(Number);
      const higherArrived = prioRule === 'lower'
        ? allActive.some(pr => pr < curPr)
        : allActive.some(pr => pr > curPr);

      if (higherArrived) {
        queues[curPr].unshift(cur); // Return current process to the front of
its queue
        gantt.push({ id: cur.id, start, end: time });
        done = true;
        break;
      }
    }

    // If the quantum expired with no interruption, put the process back at
the end of its queue
    if (!done) {
      gantt.push({ id: cur.id, start, end: time });
      queues[curPr].push(cur);
    }
```

```
  }

  const result = procs.map(p => ({
    ...p,
    tat: p.completion - p.arrival,
    wt:  p.completion - p.arrival - p.burst
  }));

  return [result, gantt];
}


// ============================================================
// SORT BY PID — sortByPID(result)
// Sorts the result array by the number inside the Process ID.
// Example: P1 comes before P2, P3, etc. regardless of arrival order.
// Extracts the numeric part from the ID string (e.g. "P2" → 2).
// ============================================================
function sortByPID(result) {
  return result.slice().sort((a, b) => {
    const numA = parseInt(a.id.replace(/\D/g, '')); // Remove letters, keep
digits
    const numB = parseInt(b.id.replace(/\D/g, ''));
    return numA - numB;
  });
}


// ============================================================
// DISPLAY RESULTS — displayResults(result, gantt, name, desc, formulas)
// Fills in Step 4 with all output:
// 1. Algorithm name, description, and formula pills
// 2. Gantt Chart (flex-based proportional blocks)
// 3. Process Summary Table (sorted by PID)
// 4. Average WT and Average TAT boxes
// Then shows Step 4 and scrolls the page to it.
// ============================================================
function displayResults(result, gantt, name, desc, formulas) {

  // Assign a unique color index to each process ID for the Gantt chart
  const colorMap = {};
  let colorIdx = 0;
  for (const g of gantt) {
    if (g.id !== 'IDLE' && !(g.id in colorMap)) colorMap[g.id] = colorIdx++ %
10;
  }

  // ---- Algorithm Header ----
  document.getElementById('algoLabel').textContent = name;
  document.getElementById('algoDesc').textContent  = desc;

  // Build formula pills (e.g. "WT = TAT - Burst Time")
  const formulaDiv = document.getElementById('algoFormula');
  formulaDiv.innerHTML = '';
  for (const f of formulas) {
    const pill = document.createElement('span');
    pill.className   = 'formula-pill';
    pill.textContent = f;
    formulaDiv.appendChild(pill);
  }

  // ---- Result Table ----
  // Sorted by PID so output always shows P1, P2, P3... in order
  const sorted = sortByPID(result);
  const tbody  = document.getElementById('resultBody');
  tbody.innerHTML = '';
  let totalWT = 0, totalTAT = 0;

  for (const p of sorted) {
```

```
    const row = document.createElement('tr');
    row.innerHTML = `
      <td><strong>${p.id}</strong></td>
      <td>${p.arrival}</td>
      <td>${p.burst}</td>
      <td>${p.priority}</td>
      <td>${p.wt}</td>
      <td>${p.tat}</td>
    `;
    tbody.appendChild(row);
    totalWT  += p.wt;
    totalTAT += p.tat;
  }

  // Compute and display average WT and TAT in the two boxes below the table
  const avgWT  = (totalWT  / sorted.length).toFixed(2);
  const avgTAT = (totalTAT / sorted.length).toFixed(2);
  document.getElementById('avgBoxes').innerHTML = `
    <div class="avg-card">
      <div class="avg-title">Average Waiting Time</div>
      <div class="avg-value">${avgWT}</div>
    </div>
    <div class="avg-card">
      <div class="avg-title">Average Turnaround Time</div>
      <div class="avg-value">${avgTAT}</div>
    </div>
  `;

  // ---- Gantt Chart ----
  // Uses CSS flex proportions so blocks always fit the card width perfectly.
  // Each block gets style.flex = its duration (number of time units it ran).
  // The browser divides the total width proportionally — no pixel math
needed.
  const ganttDiv = document.getElementById('ganttChart');
  ganttDiv.innerHTML = '';

  const chart     = document.createElement('div'); chart.className = 'gantt-
chart';
  const blocksRow = document.createElement('div'); blocksRow.className =
'gantt-blocks';
  const timesRow  = document.createElement('div'); timesRow.className  =
'gantt-times';

  for (const g of gantt) {
    const duration = g.end - g.start; // How many time units this block ran

    // Colored block — width is set by flex = duration
    const block = document.createElement('div');
    block.className   = g.id === 'IDLE' ? 'gantt-block idle' : `gantt-block
color-${colorMap[g.id]}`;
    block.style.flex  = String(duration); // Proportional width
    block.textContent = g.id;
    blocksRow.appendChild(block);

    // Time label below the block — same flex so it aligns with the block's
left edge
    const tl = document.createElement('div');
    tl.className   = 'gantt-time';
    tl.style.flex  = String(duration);
    tl.textContent = g.start;
    timesRow.appendChild(tl);
  }

  // Final end-time label — no flex, sits at the far right after the last
block
  const endLabel = document.createElement('div');
  endLabel.className   = 'gantt-time gantt-time-end';
  endLabel.textContent = gantt[gantt.length - 1].end;
  timesRow.appendChild(endLabel);
```

```
  chart.appendChild(blocksRow);
  chart.appendChild(timesRow);
  ganttDiv.appendChild(chart);

  // Show Step 4 and update the progress pills
  setActivePill(4);
  show('step4');
  document.getElementById('step4').scrollIntoView({ behavior: 'smooth' });
}


// ============================================================
// UTILITY: show(id) and hide(id)
// Show or hide any HTML section by its element ID.
// Works by adding/removing the "hidden" CSS class.
// ============================================================
function show(id) { document.getElementById(id).classList.remove('hidden'); }
function hide(id) { document.getElementById(id).classList.add('hidden'); }


// ============================================================
// GO TO STEP 1 — goToStep1()
// Called by the "Generate New Processes" button.
// Hides everything except Step 1, clears all input values,
// and scrolls back to the top to start fresh.
// ============================================================
function goToStep1() {
  hide('step4');
  hide('step2');
  hide('step3');

  // Clear the number of processes input and the process table
  document.getElementById('numProcesses').value = '';
  document.getElementById('processBody').innerHTML = '';

  setActivePill(1);
  document.getElementById('step1').scrollIntoView({ behavior: 'smooth' });
}


// ============================================================
// RESET ALL — resetAll()
// Called by the "Run Another Algorithm" button.
// Hides Step 4 and resets the algorithm selection area,
// then scrolls back to Step 3 so the user can pick a new algorithm
// with the same process data already entered.
// ============================================================
function resetAll() {
  hide('step4');
  hide('quantumRow');
  hide('prioDropdown');
  hide('prioRuleBox');

  pendingAlgo     = null;
  pendingPrioAlgo = null;

  // Remove the active highlight from all algorithm buttons
  document.querySelectorAll('.algo-btn').forEach(b =>
b.classList.remove('active-algo'));

  setActivePill(3);
  document.getElementById('step3').scrollIntoView({ behavior: 'smooth' });
}
```